Developer Guide

Acknowledgements

- **AddressBook-Level3 (AB3)**: Original codebase structure and documentation layout adapted for our project.
- **Libraries Used**:
  - JUnit 5 for unit testing and automated test execution.
- **Documentation & Tools**:
  - Mermaid for rendering UML class and sequence diagrams.
  - Gradle and Checkstyle for building, testing and enforcing code style.
- **OpenAI Codex**: I used it to help with my listing, search, sort, and menu work, along with related tests, documentation, and my portfolio page. (Beal Ng)

Design & implementation

Architecture Overview

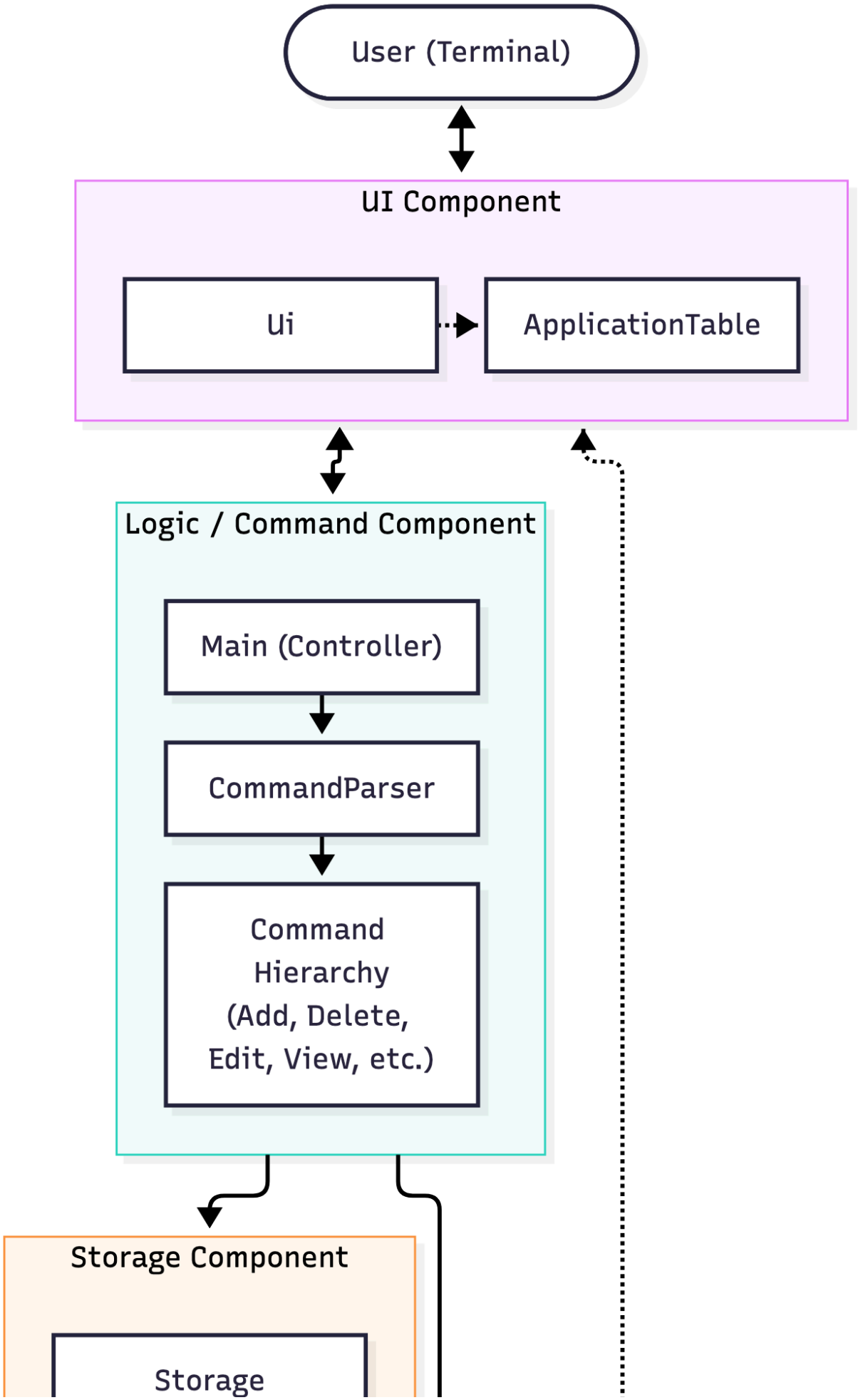
Implemented by: Siew Hean

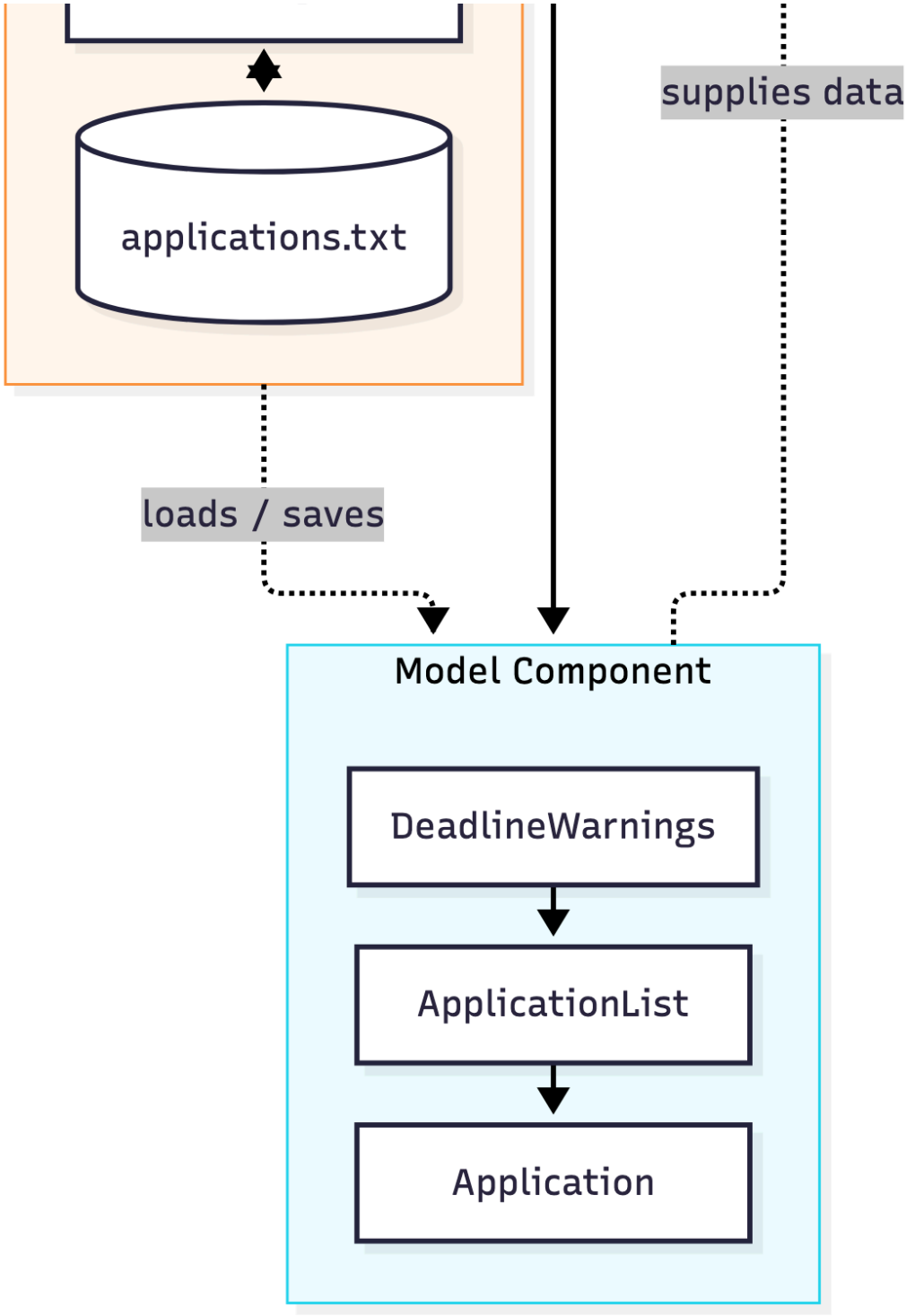
JobSeek is a command-line desktop application for managing internship and job applications. The application architecture is organized into five primary components:

- **`Main`**: Acts as the central coordinator and entry point. It initializes core dependencies, checks deadline alerts, parses user input into commands, and manages the execution loop.
- **`UI`**: Consists of `Ui` and `ApplicationTable`. It reads console inputs, renders formatted tables, application cards, deadline warning banners, and error notifications.
- **`Logic / Command`**: Built upon the Command pattern. Encapsulates all executable actions (`AddCommand`, `DeleteCommand`, `EditCommand`, `ListCommand`, etc.) inheriting from the common abstract `Command` base class.
- **`Model`**: Comprises `ApplicationList`, `Application`, and `DeadlineWarnings`. Represents in-memory domain entities, their validation rules, and lifecycle stages (`Applied`, `Online Assessment`, `Interview`, `Offer`, `Accepted`, `Rejected`, `Withdrawn`, and completion flags).
- **`Storage`**: Encapsulated by `Storage`. Handles file encoding/decoding and persistent reading/writing to `data/applications.txt`.

Architecture Diagram

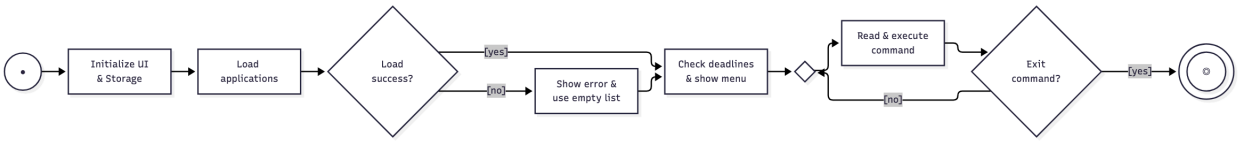
The high-level interaction between the user, UI, logic controller, domain model, and storage subsystem is shown below:


User (Terminal)
UI Component
Ui
ApplicationTable
Logic / Command Component
Main (Controller)
CommandParser
Command Hierarchy (Add, Delete, Edit, View, etc.)
Storage Component
Storage



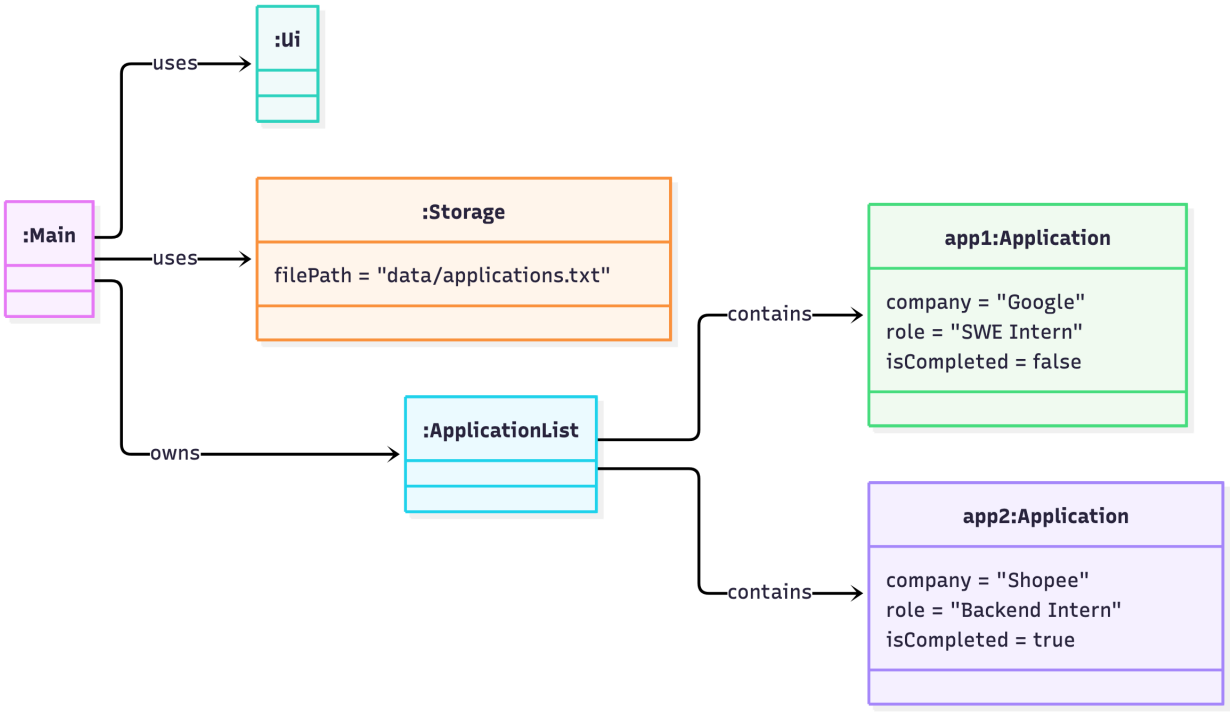
Application Lifecycle Activity Diagram

The control flow from startup, file loading, deadline inspection, command execution, and termination is illustrated in the activity diagram below:



Runtime Object Structure (Object Diagram)

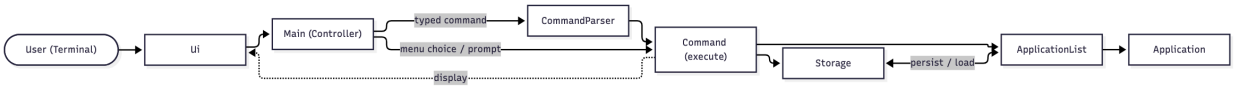
The following object diagram illustrates a concrete runtime snapshot of JobSeek after two applications have been loaded and tracked in memory:



Architecture and Command pattern

Implemented by: Siew Hean

JobSeek is a single-user command-line application. `Main` reads one line at a time from `Ui`, turns it into a `Command`, and runs it. Every action inherits from the abstract `Command` class and implements the polymorphic `execute(ApplicationList, Ui, Storage)` method. This is the Command pattern: `Main` (which dispatches) does not need to know what each command does.

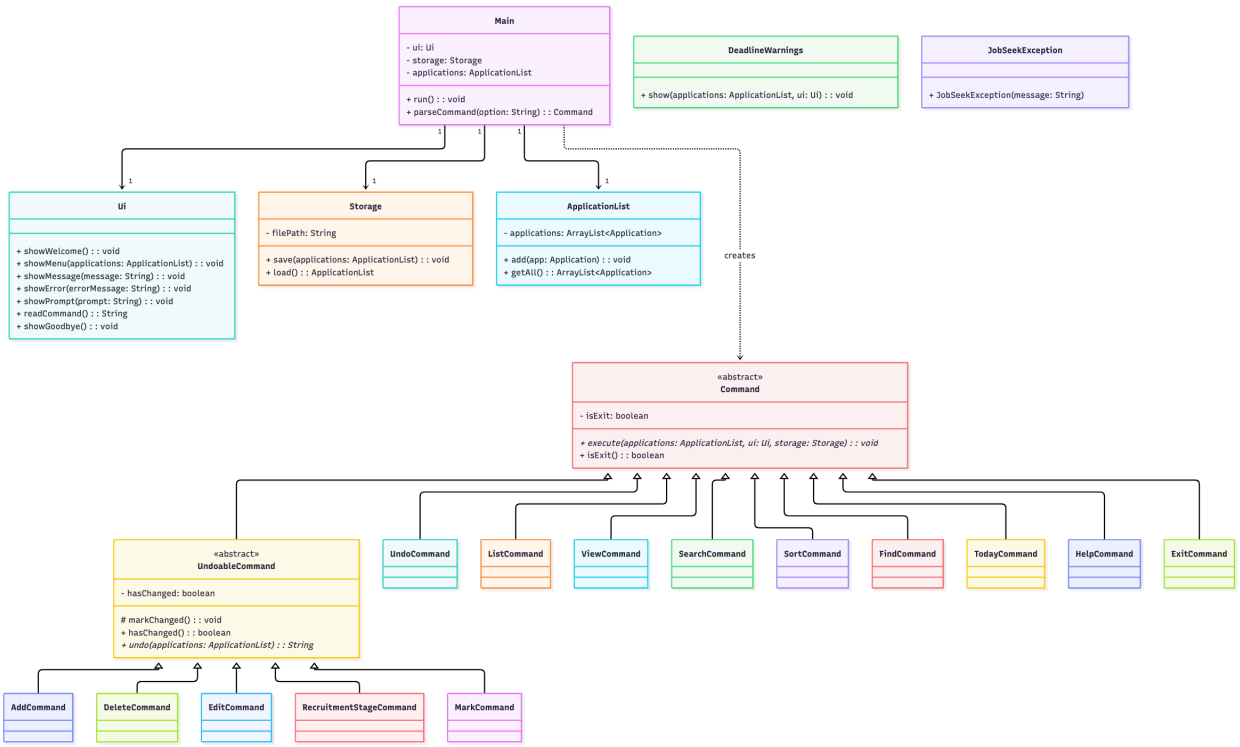


Component	Package	Responsibility
`Main`	`jobseek`	Startup, the read-parse-execute loop, the numbered menu prompts, and the undo history.
`CommandParser`	`jobseek.parser`	Turns one-line typed commands such as `add c/Google r/SWE` into `Command` objects.
`Command` and subclasses	`jobseek.command`	One class per user action.
`ApplicationList`, `Application`, `RecruitmentStage`, `ApplicationFilters`	`jobseek.model`	The in-memory data and the rules about it.
`Storage`	`jobseek.storage`	Saves and loads `data/applications.txt`.
`Ui`	`jobseek.ui`	All console input and output.
`DeadlineWarnings`	`jobseek`	Builds the warning summary shown at startup.

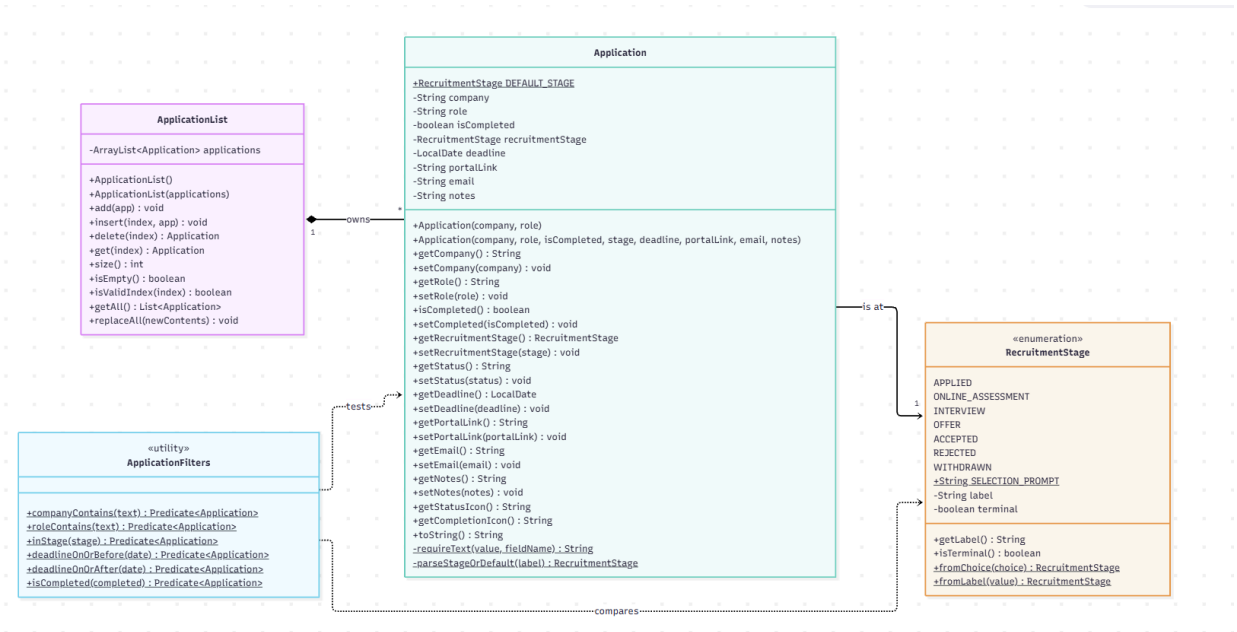
Command hierarchy. Commands that change the list extend `UndoableCommand`, which adds `undo` (see Delete confirmation and undo). Commands that only display information extend `Command` directly.

Overall Architecture Class Diagram

The class diagram below illustrates the structural relationships, associations, dependencies, and explicit UML multiplicities across the core components (`Main`, `Ui`, `Storage`, domain model, and `Command`):



Data model. `ApplicationList` owns the `Application` objects. `RecruitmentStage` is an enum, so an application can only be in one of the known stages.



Encapsulation in the model. `ApplicationList` never hands out its internal list. `getAll()` returns an unmodifiable view, so callers such as `Storage` can read the applications but cannot add, remove or reorder them. The constructor copies the collection it is given. Code that has to restore an earlier state

(`ApplicationListSnapshot`, used to roll back a failed save of an undo) calls the explicit `replaceAll(...)` operation instead of editing the list in place. The `Application` objects inside are still mutable, because that is how edits work.

Non-null entries. The collection constructor, `add`, `insert` and `replaceAll` reject null application entries with `NullPointerException`, even when assertions are disabled. Collections are copied and validated before replacing any existing content, so a rejected replacement preserves the original applications and their order. For compatibility, a null collection passed to the constructor still creates an empty list; `replaceAll(null)` is rejected.

Validation lives in the model. `Application` rejects a null or blank company or role with an `IllegalArgumentException` in its constructor and setters. This is an ordinary check, not a Java `assert`, because assertions are disabled when users run the JAR. "Blank" is defined with `String.isBlank()` / `strip()`, which know Unicode spaces (such as U+2003 EM SPACE), and the same definition is used by the prompts, `CommandParser`, `AddCommand`, `EditCommand` and `Storage`. Using `trim()` somewhere and `isBlank()` elsewhere is how a name that could be added once vanished after a restart. The commands still validate first so that users see a friendly error message instead of an exception.

Add application feature

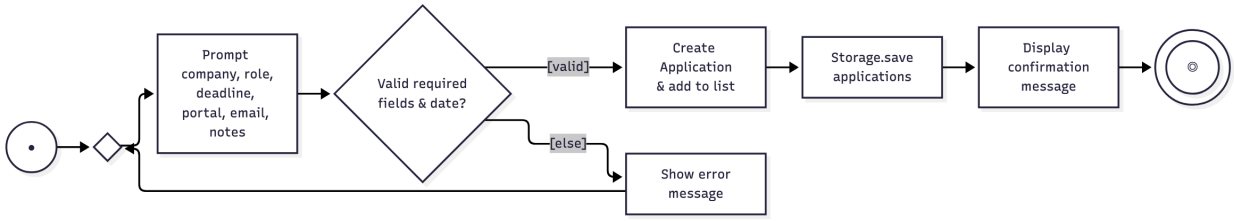
Implemented by: Siew Hean

The `AddCommand` allows job seekers to add a new internship or job record with a company, a role and an optional deadline. It can be reached from menu option `1` or from the typed command `add c/COMPANY r/ROLE [d/YYYY-MM-DD]`.

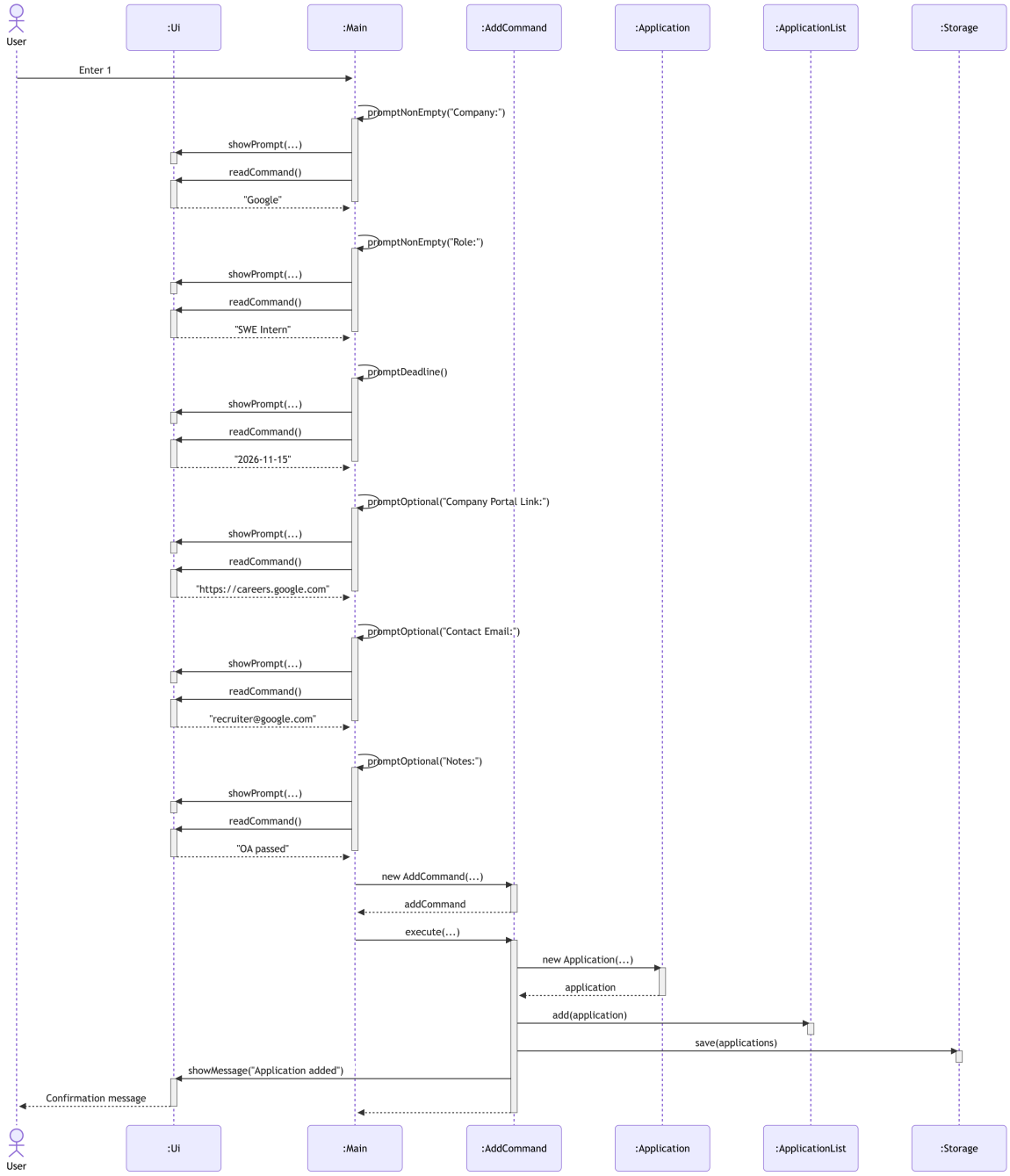
- **Workflow (menu option `1`)**:
  1. `Main.handleMenuAdd()` queries the user for non-empty company and role strings using `promptNonEmpty()`, followed by optional deadline (`promptDeadline()`), portal link, contact email, and notes (`promptOptional()`). Blank or whitespace-only inputs for required fields are rejected with informative error messages and reprompted.
  2. Once inputs are obtained, `Main` instantiates `AddCommand(company, role, deadline, portalLink, email, notes)`. (For the typed command, `CommandParser.parseAdd()` builds an `AddCommand`.)
  3. `AddCommand.execute()` instantiates a new `Application` with all specified attributes, initializing completion status to `false` (`[ ]`) and recruitment stage to `Applied` (`Application.DEFAULT_STAGE`).
  4. The application is added to `ApplicationList` and remembered by the command so it can be undone. Internal assumptions (such as list size incrementing by 1 and target non-null) are verified via Java `assert`.
  5. The updated list is persisted to disk via `Storage.save(applications)`, and confirmation displaying recorded details is rendered to the user via `Ui.showMessage()`.

Add Application Activity Diagram

The user input validation, prompt looping, and execution lifecycle for adding an application is illustrated in the activity diagram below:



Add Application Sequence Diagram



View application feature

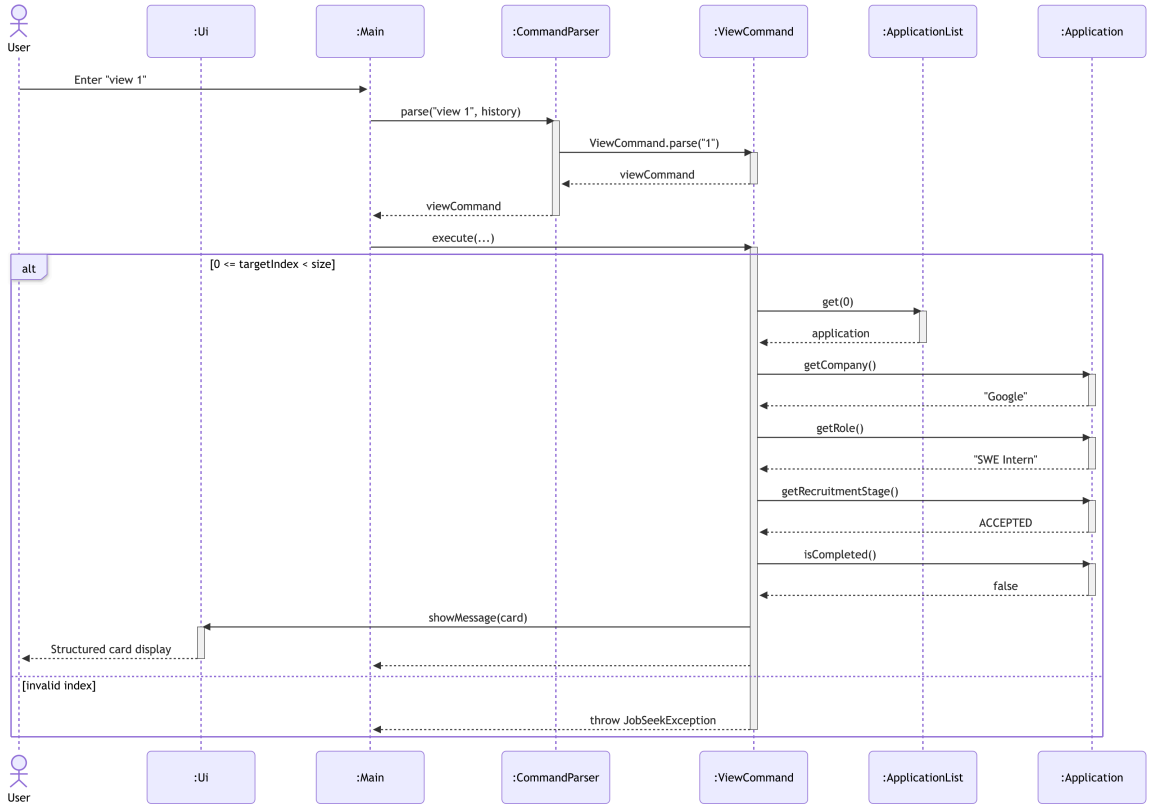
Implemented by: Peng Bohao

Overview

The `ViewCommand` displays all recorded information for an individual application as a structured, visual card. It is executed via the `view <application_number>` standalone command.

Design and implementation

- **Parsing**: `ViewCommand.parse(String args)` parses user input following the `view` command prefix. It validates that the argument is a non-empty, positive integer (`1`-based). If the argument is invalid or non-numeric, it throws a `JobSeekException` with helpful usage guidance (`Please provide a valid positive application number (e.g. 'view 1').`).
- **Execution**:
  1. `ViewCommand.execute(ApplicationList, Ui, Storage)` validates that the zero-based target index satisfies `0 <= targetIndex < applications.size()`.
  2. If the index is out of bounds, it throws a `JobSeekException` reporting the invalid index and the current list size (e.g., `Application index 5 does not exist. (Total applications: 3)`).
  3. Upon successful bounds validation, it retrieves the target `Application` via `applications.get(targetIndex)`.
  4. It constructs a visual card bordered with dividers, displaying the application's company, role, recruitment stage (`Applied`, `Online Assessment`, `Interview`, `Offer`, `Accepted`, `Rejected`, `Withdrawn`), and completion status (`Yes` or `No`).
  5. The card is rendered via `Ui.showMessage()`.
  6. Because `ViewCommand` is a read-only operation, it does not modify the application list or trigger disk persistence.



Design considerations

- **Card view vs tabular view**:
  - *Current choice*: A structured card view focuses on one record at a time, providing space to cleanly display all recorded attributes (including recruitment stage and completion status) without line truncation or horizontal overflow.
  - *Alternative*: Adding more columns to the standard list table. This was rejected because widening the table reduces readability on smaller terminal windows and clutters the overview.
- **Read-only decoupling**:
  - `ViewCommand` adheres to the Command pattern without calling `Storage.save()`, ensuring zero disk I/O overhead.

Filtered application listing feature

Implemented by: Peng Bohao, Beal Ng

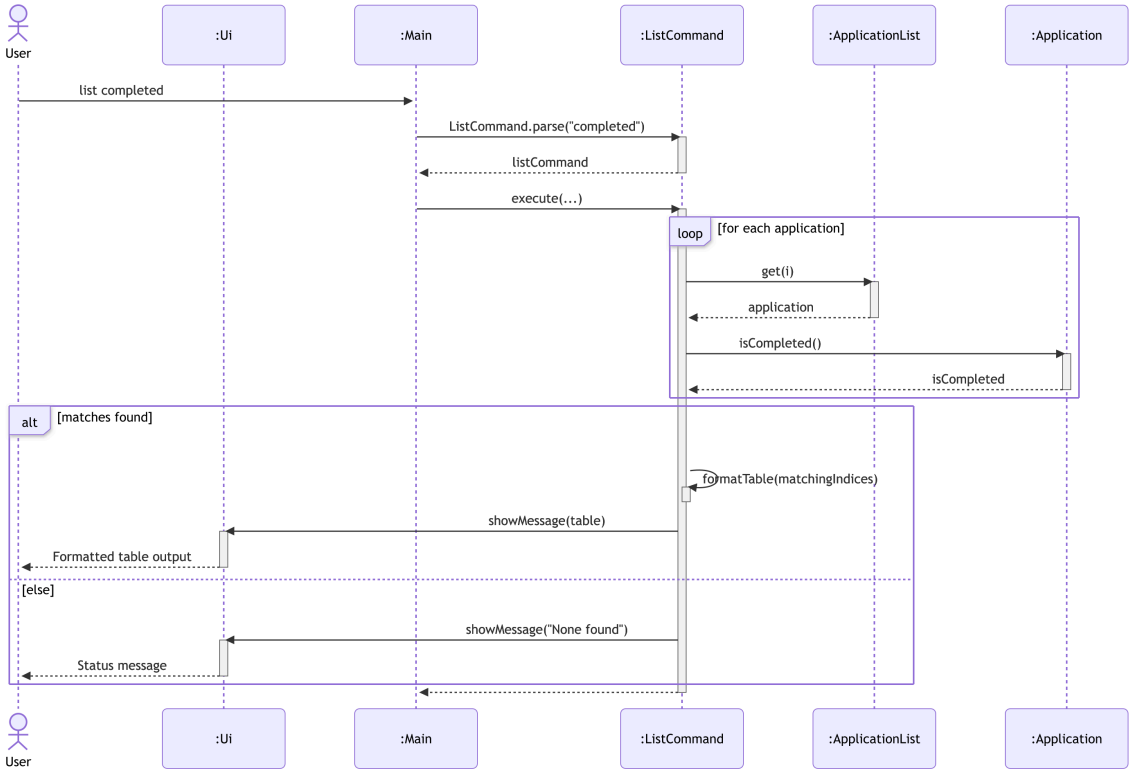
Overview

The `ListCommand` presents applications in an aligned ASCII table. It supports completion-status filtering so users can focus on active applications, completed applications, or inspect all tracked opportunities.

Design and implementation

- **Filter criteria**: Controlled by the `FilterType` enum:
  - `ACTIVE`: Displays only uncompleted applications (`isCompleted() == false`).
  - `COMPLETED`: Displays only completed applications (`isCompleted() == true`).
  - `ALL`: Displays all applications regardless of completion state.
- **Triggering mechanisms**:
  - Standalone text commands: `list` (defaults to `ACTIVE`), `list completed`, `list all`. Parsed by `ListCommand.parse(String args)`. Unrecognized filters throw a `JobSeekException` with usage instructions.
  - Numeric menu options: Menu option `2` (`List all applications`) instantiates `ListCommand(FilterType.ALL)`, and menu option `3` (`List uncompleted applications`) instantiates `ListCommand(FilterType.ACTIVE)`.
- **Workflow**:
  1. `ListCommand.execute()` checks if `applications.isEmpty()`. If the list is empty, it outputs `"No applications found."`.
  2. Iterates over all applications and collects the zero-based indices of matching records into an `ArrayList<Integer> matchingIndices` using `matchesFilter(app)`.
  3. If no records match the chosen filter, context-sensitive feedback is displayed:
     - `COMPLETED`: `"No completed applications found."`
     - `ACTIVE`: `"No active applications found."`
     - `ALL`: `"No applications found."`
  4. Dynamically calculates column widths for `No.`, `Completed`, `Company`, `Role`, `Stage`, `Deadline`, `Portal Link`, `Email`, and `Notes` based on the longest values among matching entries, guaranteeing clean tabular alignment.

5. Renders each matching row, preserving the application's original 1-based index (`index + 1`).
6. Displays the formatted table via `Ui.showMessage()`.



Design considerations

- **Preserving original 1-based indices**:
  - *Current choice*: The `No.` column displays the application's original index from `ApplicationList` rather than re-indexing from 1 in the filtered view. This guarantees that numbers remain consistent across `list`, `list completed`, `view`, `mark`, `edit`, and `delete` commands.
  - *Alternative*: Re-numbering matching rows sequentially (1, 2, ...). This was rejected because it introduces confusion when users attempt to execute `view` or `mark` on an application whose display number differs from its list position.
- **Default to active applications**:
  - Job seekers primarily need to review and act upon active, ongoing applications. Concluded applications are hidden by default to keep the interface concise and relevant.

Mark and unmark application completion feature

Implemented by: Peng Bohao

Overview

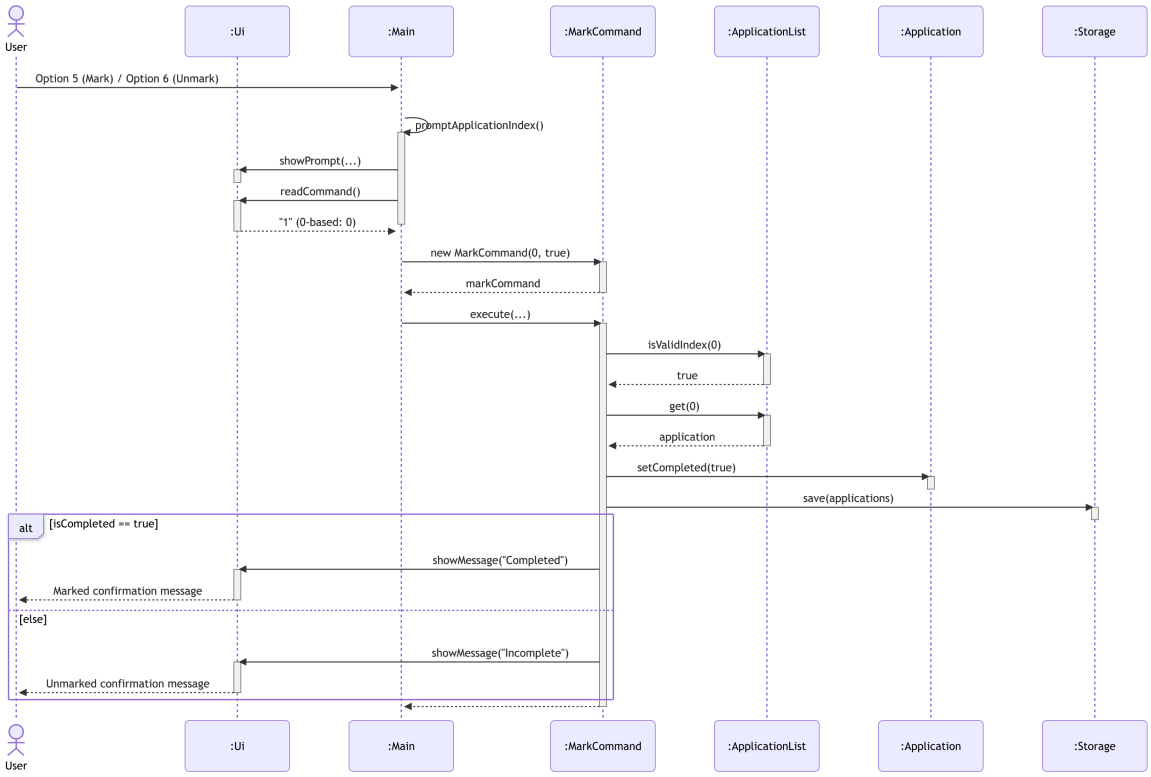
The `MarkCommand` allows users to toggle an application's lifecycle between active/incomplete (`[ ]`) and completed (`[X]`). Completed applications are typically archived from the default active list.

Design and implementation

- **Menu triggers**:

- Menu option `5`: Calls `handleMenuMark(true)` to mark an application as completed.
- Menu option `6`: Calls `handleMenuMark(false)` to unmark an application.

- **Workflow**:
  1. `Main.promptApplicationIndex()` prompts the user for the 1-based application index and converts it to a zero-based integer.
  2. `Main` instantiates `MarkCommand(targetIndex, isCompleted)`.
  3. `MarkCommand.execute(applications, ui, storage)` checks whether `applications.isEmpty()`. If empty, it displays an error message via `Ui.showError()`.
  4. Validates that `targetIndex` is valid using `applications.isValidIndex(targetIndex)`. If invalid, displays a bounds error (e.g., `Invalid application index: 4. Please enter a number between 1 and 2.`).
  5. If the application already has the requested state, shows an "already marked" message and returns without saving, so the command is not recorded for `undo`. Otherwise it updates the completion state using `application.setCompleted(isCompleted)`.
  6. Calls `saveOrRollback(applications, storage)` to persist the change, restoring the previous completion flag if saving fails.
  7. After a successful save, displays a confirmation message showing the updated application representation:
     - Marked: `"Nice! I've marked this application as completed:\n " + application`
     - Unmarked: `"OK, I've marked this application as not completed yet:\n " + application`



Design considerations

- **Single unified command vs separate commands**:
  - *Current choice*: A unified `MarkCommand` parameterized with boolean `isCompleted`. The workflow for validation, index checking, list mutation, and disk persistence is identical for both actions;

unifying them eliminates code duplication.
    - *Alternative*: Separate `MarkCommand` and `UnmarkCommand` classes. This was rejected to avoid redundant boilerplate and maintenance overhead.

- **Immediate disk persistence**:
    - Changes to completion status are saved before success is reported. A failed save restores the previous in-memory flag. Storage uses an atomic move where supported, with a non-atomic compatibility fallback; see Saving applications for its guarantees and limitations.

Delete application feature

Implemented by: Chua Jinghao, Peng Bohao

Overview

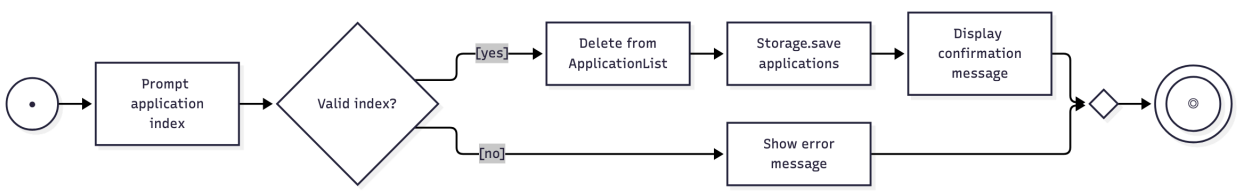
The `DeleteCommand` allows users to permanently remove an application by its 1-based index number (as shown in the list table). Removing an application triggers immediate list re-indexing and disk synchronization.

Design and implementation

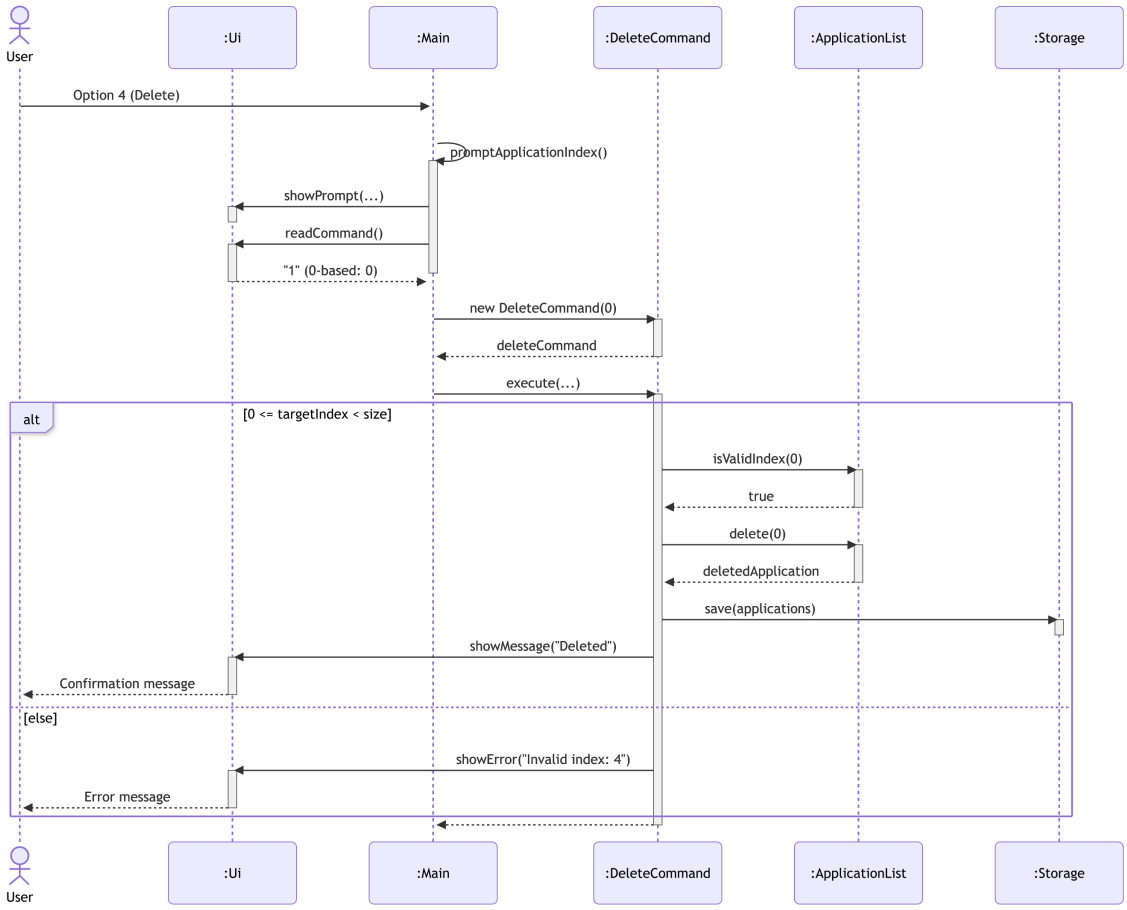
- **Trigger**: Menu option `4` invokes `Main.handleMenuDelete()`.
- **Workflow**:
    1. `Main.promptApplicationIndex()` checks whether `applications.isEmpty()`. If empty, an error is displayed and execution terminates.
    2. Prompts the user for the 1-based index and converts it to a 0-based integer.
    3. `DeleteCommand.execute()` validates index bounds via `applications.isValidIndex(targetIndex)`.
    4. Requests confirmation; only `y` or `yes` proceeds. Then `applications.delete(targetIndex)` removes the entry and renumbers following elements.
    5. Retains the removed object and its index for undo or rollback; the object is not destroyed.
    6. Calls `saveOrRollback(applications, storage)`. If saving fails, the same object is reinserted at its original position and the command is not recorded in history.
    7. A confirmation message displaying the removed entry is shown via `Ui.showMessage()`.

Delete Application Activity Diagram

The decision flow, bounds validation, and deletion logic is shown in the activity diagram below:



Delete Application Sequence Diagram



Edit application and recruitment stage tracking

Implemented by: Siew Hean

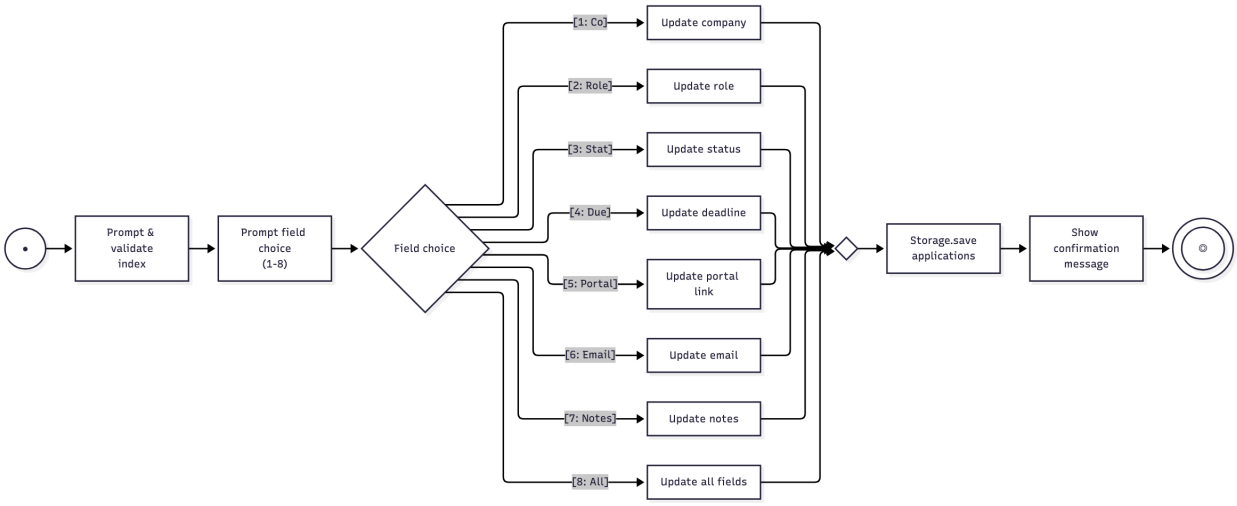
The edit feature allows users to modify existing applications without deleting and re-entering them. Recruitment progress is represented by the `RecruitmentStage` enum (`Applied`, `Online Assessment`, `Interview`, `Offer`, `Accepted`, `Rejected`, `Withdrawn`). The enum owns each display label and whether a stage is terminal. Users may edit a stage freely to handle corrections and different company processes.

- **Design and Implementation**:
  - `EditCommand` accepts a zero-based target index and nullable field replacements: `newCompany`, `newRole`, `newStage`, `newDeadline`, `newPortalLink`, `newEmail`, and `newNotes`. A `null` value indicates that the corresponding field should remain unchanged.
  - `RecruitmentStageCommand` provides direct, dedicated recruitment stage modification for targeted application records. The menu uses it when only the stage is edited.
  - In `Main.handleMenuEdit()`, the user chooses the target application index, followed by choosing a field from the structured edit menu (`Ui.showEditMenu()`, choices 1 to 8):
    - `1`: Company only (`EditCommand`)
    - `2`: Role only (`EditCommand`)
    - `3`: Recruitment stage only (`RecruitmentStageCommand`)
    - `4`: Deadline only (`EditCommand`)
    - `5`: Company Portal Link only (`EditCommand`)

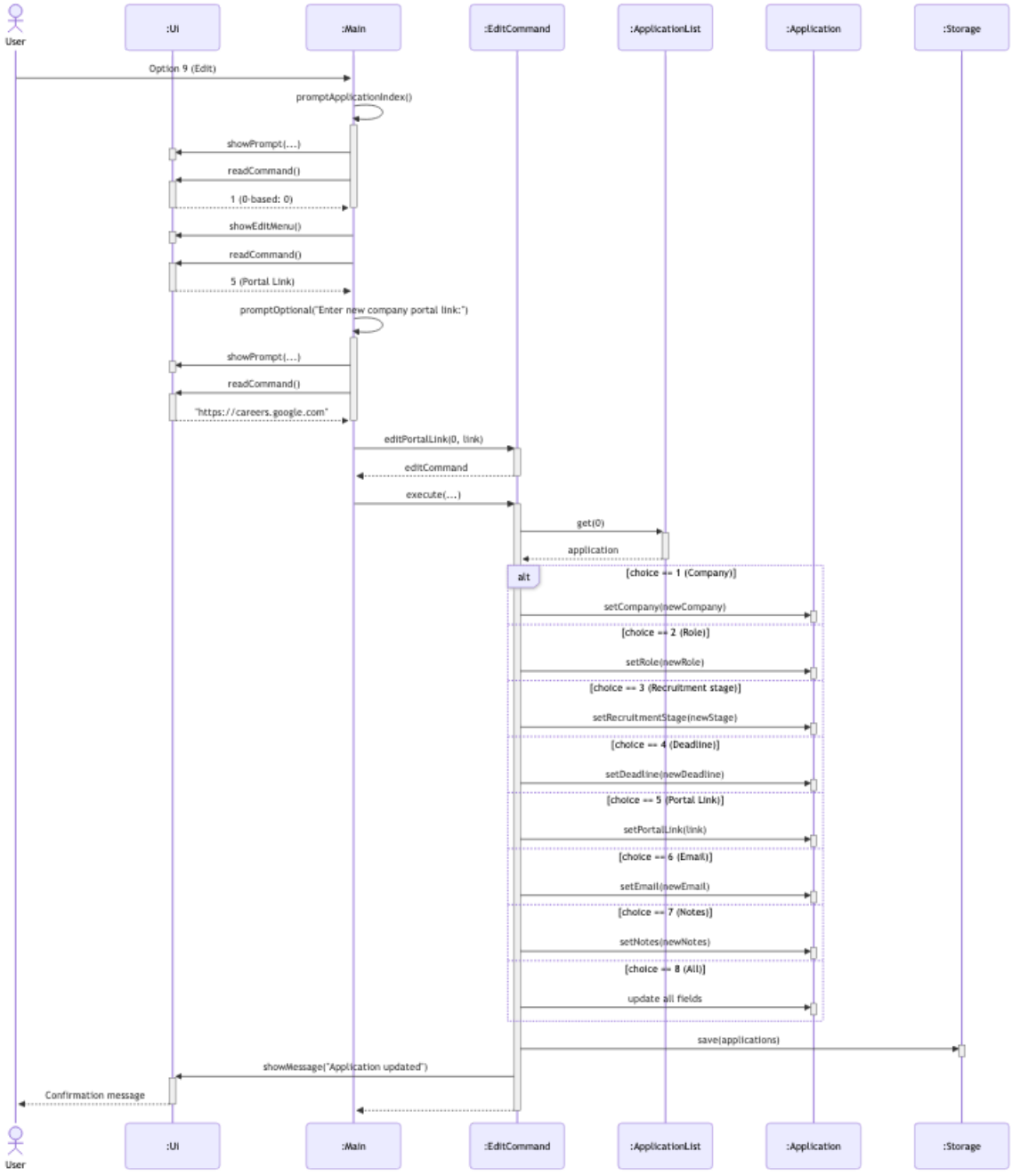
    - `6` : Contact Email only (`EditCommand`)
    - `7` : Notes only (`EditCommand`)
    - `8` : All fields in sequence (Company, Role, Recruitment Stage, Deadline, Portal Link, Email, Notes) (`EditCommand`)
- The typed command `edit INDEX [c/..] [r/..] [s/..] [d/..] [p/..] [e/..] [n/..]` is parsed by `CommandParser.parseEdit()` and always creates an `EditCommand` .
- Stage input normalization in `Main.promptRecruitmentStage()` accepts numbered choices (`1` for Applied, `2` for Online Assessment, `3` for Interview, `4` for Offer, `5` for Accepted, `6` for Rejected, `7` for Withdrawn) and case-insensitive stage names (e.g. `OA`). The legacy label `Pending` is read as `Applied` .
- Optional fields (deadline, portal link, email, notes) can be updated or cleared with blank inputs.
- Both commands extend `UndoableCommand` and remember previous values, so `undo` can restore them.
- **Defensive programming & Assertions**: Java `assert` statements verify preconditions (non-null arguments, in-bounds target indices) and postconditions (non-empty required fields, updated stage).

Edit Application Activity Diagram

The user interaction and validation workflow for updating application fields is illustrated in the activity diagram below:



Edit Application Sequence Diagram



Help feature

Implemented by: Siew Hean

`HelpCommand` displays a summary of the numeric menu options (`[0]` to `[10]`) and of the typed commands (`list` , `undo` , `add` , `edit` , `delete` , `mark` , `unmark` , `find` , `search` , `sort` , `today` , `view`). It extends `Command` and does not change any data or write to disk. Because the help text is a hard-coded string, it must be updated whenever a command is added; `HelpCommandTest` checks for the main entries.

Menu layout and command validation

Implemented by: Beal Ng

`Ui.showMenuOptions()` prints options `1` to `10` in five paired rows, with `0` for Exit below them. This shortens the home menu without changing how `Main` handles any option number. `UiTest` checks the displayed pairs.

For typed commands that take no arguments (`sort`, `help`, `today`, `undo`, and `exit`), `CommandParser.requireNoArguments()` rejects any trailing text with a `JobSeekException` and the command's usage. `Main` displays the error instead of running the command. `CommandParserTest` covers all five commands with extra text.

Search and alphabetical views

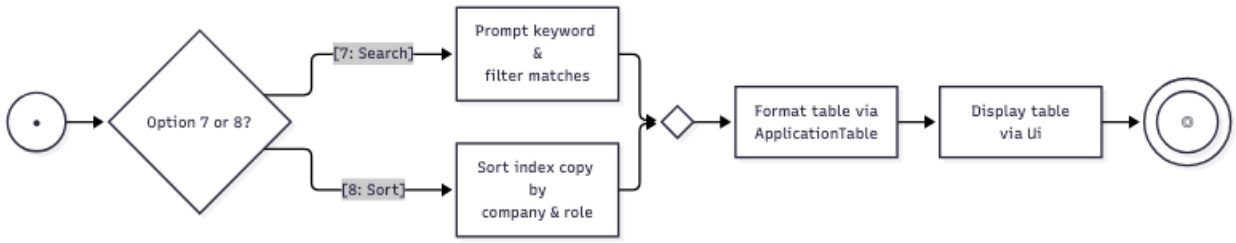
Implemented by: Beal Ng

`Main` translates numeric menu input into a `Command`. Option `7` asks for a non-blank search term, then creates `SearchCommand`; option `8` creates `SortCommand`. The typed commands `search KEYWORD` and `sort` create the same commands through `CommandParser`. For searches with several conditions, see Combined filters. Both commands operate on an `ApplicationList` but do not mutate it or call `Storage.save`.

The commands collect zero-based indices of matching applications. `SearchCommand` matches a case-insensitive substring against both company and role. `SortCommand` sorts a copy of the indices by company, then role, using a case-insensitive comparator; original index breaks ties. `ApplicationTable` formats either view with one-based numbers derived from the original indices. This keeps subsequent mark and delete commands unambiguous, even when the display order differs from storage order.

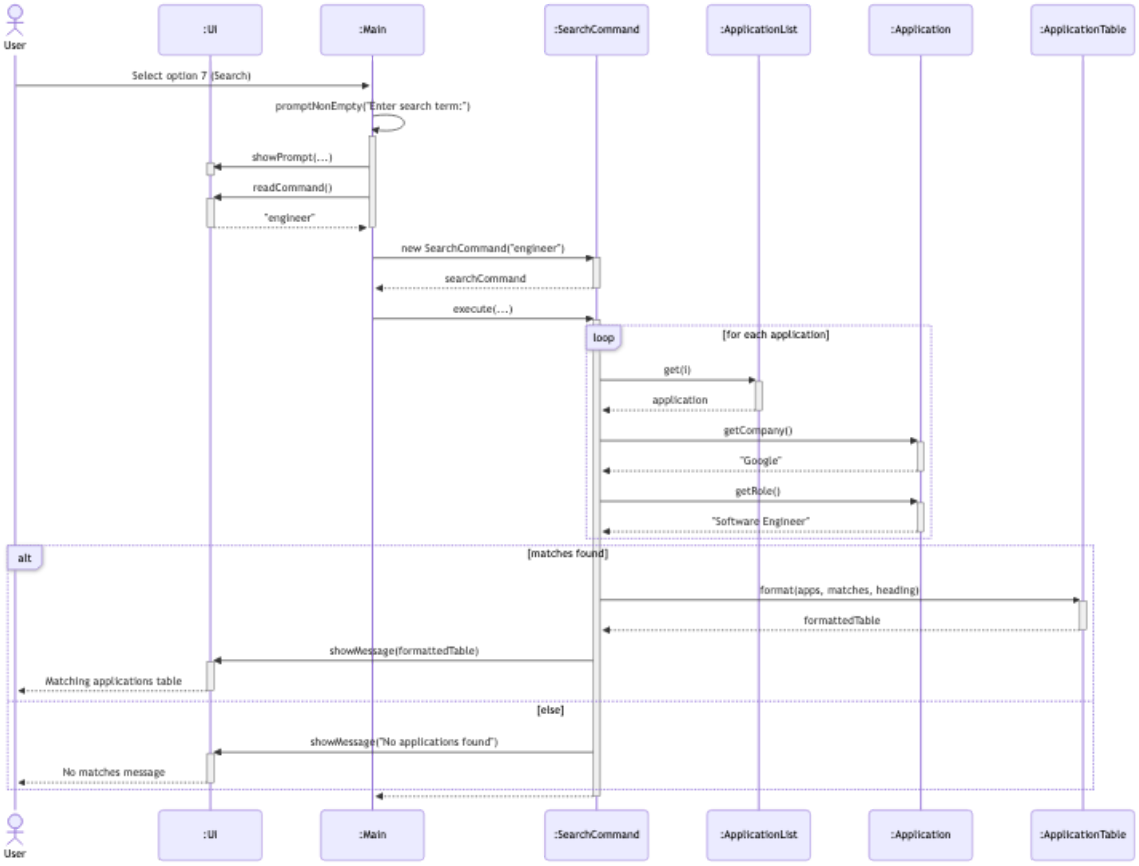
Search and Sort Activity Diagram

The flow of user input, matching/sorting logic, and display formatting for both search and sort operations is shown below:



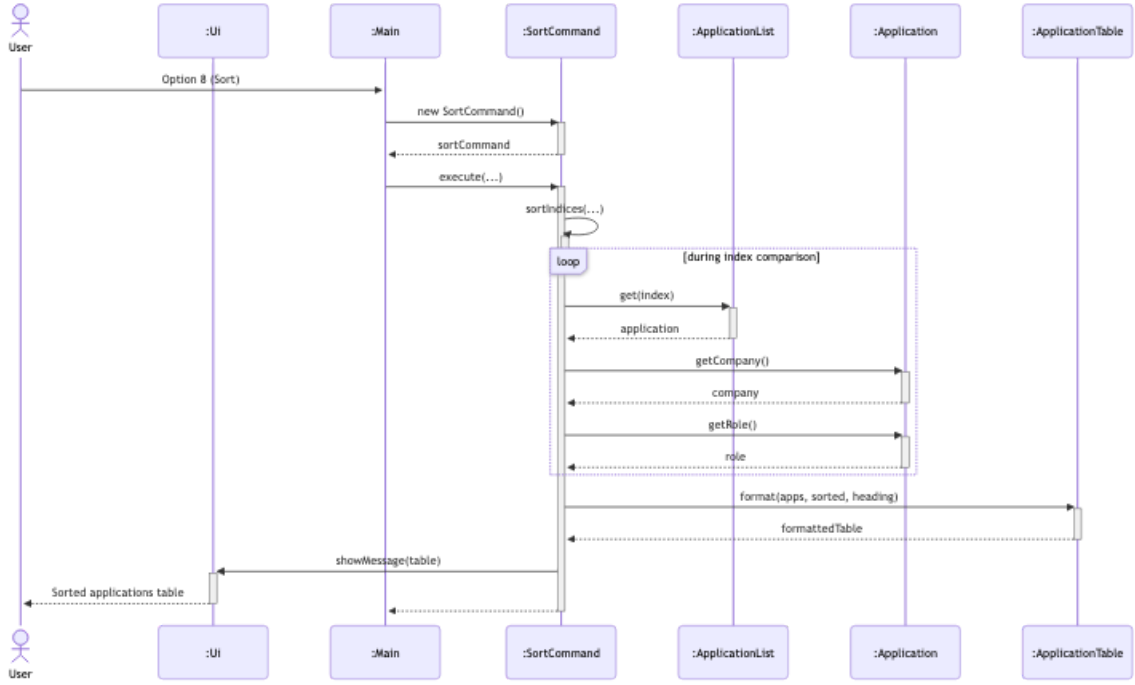
Search Sequence Diagram

The interaction sequence for searching applications is shown below:



Sort Sequence Diagram

The alphabetical view follows a comparable execution path using `SortCommand`. It sorts a copy of the index array rather than mutating storage or the underlying `ApplicationList`:



The alphabetical view follows the same formatting path after sorting the index copy. Keeping the list itself unchanged means there is no save operation and existing application numbers remain stable.

Typed command parser

Implemented by: Peng Bohao

Overview

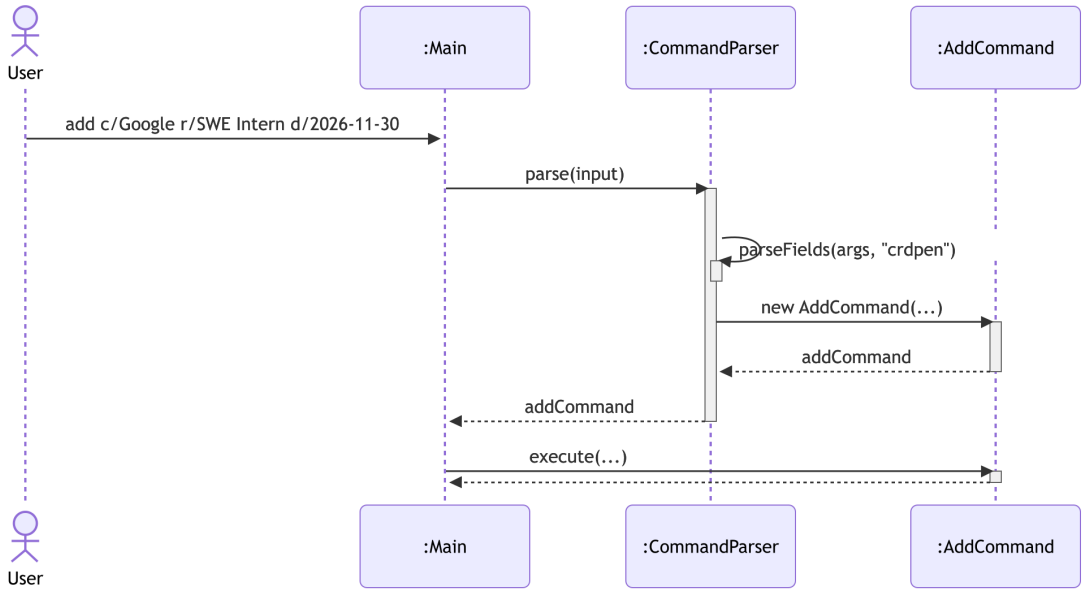
Besides the numbered menu, users can type a full command on one line, such as `add c/Google r/SWE Intern d/2026-11-30` or `delete 2`. This matches the target user, who types quickly and prefers the keyboard.

Design and implementation

`CommandParser` (package `jobseek.parser`) is a utility class providing overloaded `parse(String)` and `parse(String, CommandHistory)` methods. It serves as the single source of truth for parsing all typed user commands (Separation of Concerns and Single Responsibility Principle):

Input	Command created
`add c/.. r/.. [d/..]`	`AddCommand`
`edit INDEX [c/..] [r/..] [s/..]`	`EditCommand`
`delete INDEX`	`DeleteCommand`
`mark INDEX`, `unmark INDEX`	`MarkCommand`
`find [c/..] [r/..] [s/..] [b/..] [a/..] [m/..]`	`FindCommand`
`search KEYWORD`	`SearchCommand`
`sort`, `help`, `exit`	`SortCommand`, `HelpCommand`, `ExitCommand`
`view INDEX`	`ViewCommand`
`list [completed|all]`	`ListCommand`
`today`	`TodayCommand`
`undo`	`UndoCommand`

`Main.parseCommand` delegates all typed command parsing directly to `CommandParser.parse(trimmedOption, history)`. The parser returns `null` when the first word is not a recognized typed command (for example a numbered menu option `1` to `10`), allowing `Main` to fall back to the numbered menu switch cleanly. If the first word is a valid typed command but its syntax or arguments are invalid, the parser throws a `JobSeekException` containing actionable usage guidance, which `Main` catches and presents through `Ui.showError()`.



Fields are found with a regular expression such as `(?:^|\s)([crsdpenbam])/`, which matches one prefix letter and a slash at the start of the text or after whitespace. `add` and `edit` only treat letters that are prefixes in some command as prefixes, so slashes after other letters (`w/ HR`, `I/O`) stay in the value; `find` accepts any letter and reports unknown ones, because it has no free text. Everything between one prefix and the next is that field's value, so values can contain spaces. Text before the first prefix is kept separately; `edit` treats it as the index, and `add` and `find` reject it. Because the prefix must follow whitespace, slashes inside words such as `AT&T/Intel` are ordinary text. The parser also rejects prefixes that belong to a different command (for example `s/` in `add`), repeated prefixes and empty values. The exception is `edit`, where an empty `d/`, `p/`, `e/` or `n/` is valid and clears that field.

User-typed indexes are one-based, and the parser converts them to the zero-based indexes that commands expect. A stage typed after `s/` is validated with `RecruitmentStage.fromLabel` so a typo is reported as a normal error before any command runs.

Design considerations

- **Separate class vs. more code in `Main`:** `Main` already held all the menu prompting. Putting parsing in its own class keeps `Main` smaller and lets the parser be unit-tested without simulating console input (`CommandParserTest`).
- **Prefix syntax vs. positional arguments:** with `add Google SWE Intern` it is impossible to tell where the company name ends. Prefixes allow multi-word values and any field order.
- **Limitation:** a value cannot contain a space followed by one of the real prefix letters and a slash (for example `r/`). Other letters are fine.

Combined filters (`find`)

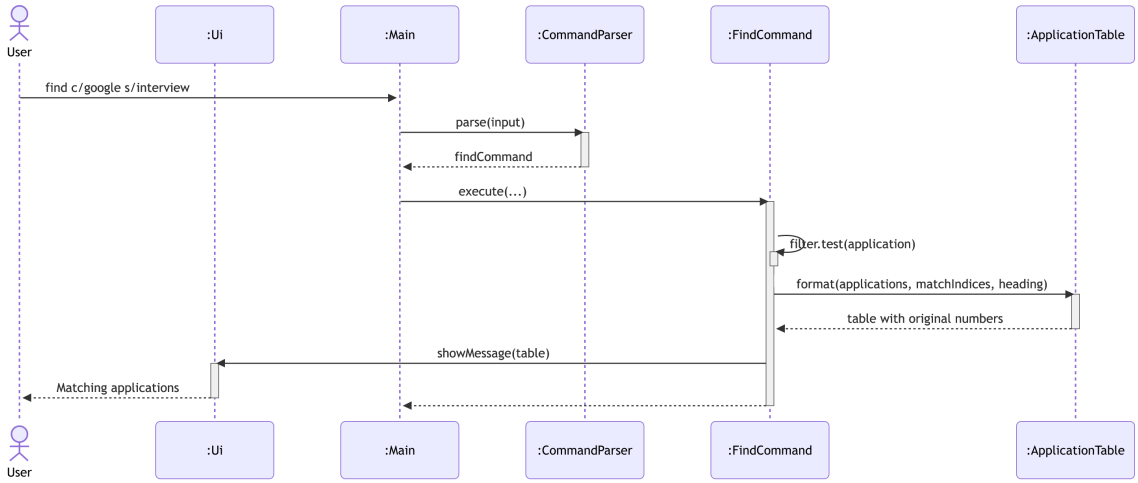
Implemented by: Peng Bohao

Overview

`find` lets the user combine several conditions, for example `find c/google s/interview a/2026-11-01`. The existing `search` command only matches one keyword against company or role, which is not enough once there are dozens of applications.

Design and implementation

Class	Responsibility
`ApplicationFilters` (`model`)	Static factory methods such as `companyContains`, `inStage` and `deadlineOnOrBefore`. Each returns a `Predicate<Application>`.
`CommandParser` (`parser`)	`parseFind` turns each prefix (`c/ r/ s/ b/ a/ m/`) into a predicate and joins them with `Predicate.and`. It also builds a readable description of the conditions.
`FindCommand` (`command`)	Holds one combined `Predicate<Application>` and a description. It tests every application, then shows the matches through `ApplicationTable`.



`FindCommand` never needs to know which filters were chosen, because the parser has already combined them into a single predicate. Adding a new filter therefore means adding one factory method to `ApplicationFilters` and one prefix to `parseFind`; `FindCommand` stays unchanged. This is the strategy pattern, using Java's functional interface `Predicate` instead of a hand-written interface. `FindCommand` is read-only, so it does not call `Storage.save`.

Design considerations

- **Predicates vs. one method with many `if` statements:** a single method checking six optional parameters grows with every new filter and is hard to test. Small predicates are tested one by one (`FindCommandTest`) and combined freely.
- **`and` only:** all filters must match. Supporting `or` would need a more complex syntax, so it is left out.
- **Applications without a deadline** never match `b/` or `a/`, because "unknown" is not before or after any date.
- **Original numbering:** results use the index in the full list (like `search` and `sort`), so numbers stay valid for `edit` and `delete`.

Delete confirmation and undo

Feature tested: Delete confirmation and undo (Implemented by: Peng Bohao)

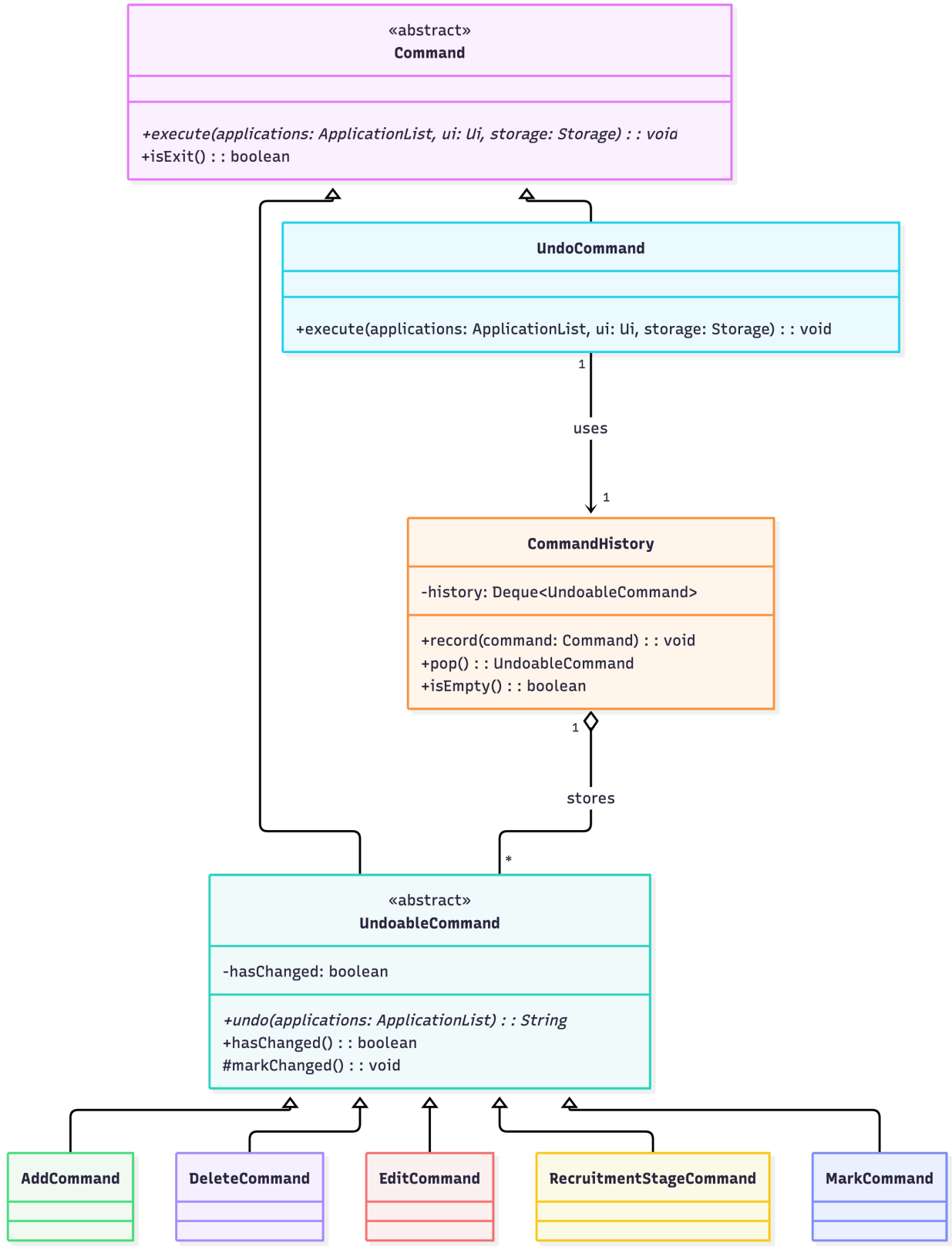
Implemented by: Peng Bohao

Overview

`DeleteCommand` asks the user to confirm before removing an application, and `undo` reverses the most recent change. Both exist to protect the user from slips: the data file is overwritten on every save, so a mistaken delete would otherwise be permanent.

Design and implementation

Class	Responsibility
`UndoableCommand`	Adds `undo(ApplicationList)`, `hasChanged()` and `saveOrRollback()` to reverse failed changes.
`AddCommand`, `DeleteCommand`, `EditCommand`, `RecruitmentStageCommand`, `MarkCommand`	Extend `UndoableCommand`. Each remembers what it needs (the affected `Application` and its old values) while executing.
`CommandHistory`	A stack (`ArrayDeque`) of undoable commands that actually changed the list.
`UndoCommand`	Peeks at the newest command, snapshots the current list, undoes and saves; pops history and reports success only after saving.
`ApplicationListSnapshot`	Restores list order, field values and original object identities if saving an undo fails.
`Main`	Owns the `CommandHistory`, records each executed command, and creates `UndoCommand` for the text command `undo`.



Undo execution sequence:

1. `UndoCommand` calls `history.peek()`; empty history produces an error without changing data.
2. It captures an `ApplicationListSnapshot` and calls the latest command's `undo(applications)`.
3. It calls `storage.save(applications)`.
4. On success, it calls `history.pop()` and displays the confirmation. On `IOException`, it restores the snapshot and retains the history entry for retry.

Confirmation happens inside `DeleteCommand.execute` by calling `ui.readCommand()`, so it applies however the command was created (menu option `4` or the typed `delete INDEX`). Only `y` or `yes` (case-insensitive) confirms; anything else, including end of input, cancels.

Undo works on the same `Application` objects that were changed. Commands keep a reference plus the previous values, so an undo is exact even when two applications look identical (`AddCommand.undo` compares by identity, not `equals`). `DeleteCommand.undo` uses `ApplicationList.insert(index, app)` to restore the original position.

Design considerations

- **Command history and temporary snapshots:** persistent in-session history stores commands and their previous values. Undo additionally takes one temporary snapshot for save-failure recovery. Restoring the original objects preserves references in earlier history entries; replacing them with copies would break subsequent undo operations.
- **`hasChanged()`:** rejected commands never call `markChanged()`, and neither do commands that would not alter anything (`EditCommand.changesAnything()` is false, `RecruitmentStageCommand` with the current stage, `MarkCommand` on an application already in that state). A failed save rolls back the change and clears this flag. None of these cases adds an entry to history.
- **Limitations:** history is in memory only and there is no redo.

Today dashboard

Implemented by: Peng Bohao

`TodayCommand` shows overdue, due-today and upcoming (next 7 days) deadlines of active applications, then a summary (active/completed counts, active applications per recruitment stage). `Main.parseCommand` creates it for the text command `today`.

It is read-only, so it does not call `Storage.save`. Its constructor accepts a `LocalDate` so tests can fix "today"; the no-argument constructor uses `LocalDate.now()`. Each entry prints the application's original one-based index so it can be used with `view`, `edit` and `delete`. The classification rules match `DeadlineWarnings` (same 7-day window, completed applications ignored), but the dashboard can be run on demand instead of only at startup.

Alternative considered: reuse `DeadlineWarnings` directly. It was not reused because it builds a single text block with no per-section counts or summary; extracting a shared classifier is a possible later refactoring.

Startup deadline warnings

Implemented by: Chua Jinghao

Overview

The startup deadline warnings feature helps users identify applications that require attention when they launch JobSeek. It displays a summary of overdue deadlines, deadlines due today, and deadlines within the next seven days.

Warnings apply only to applications that have not been marked as completed. Applications without a recorded deadline are excluded. This allows users to track applications even when their deadlines are unknown.

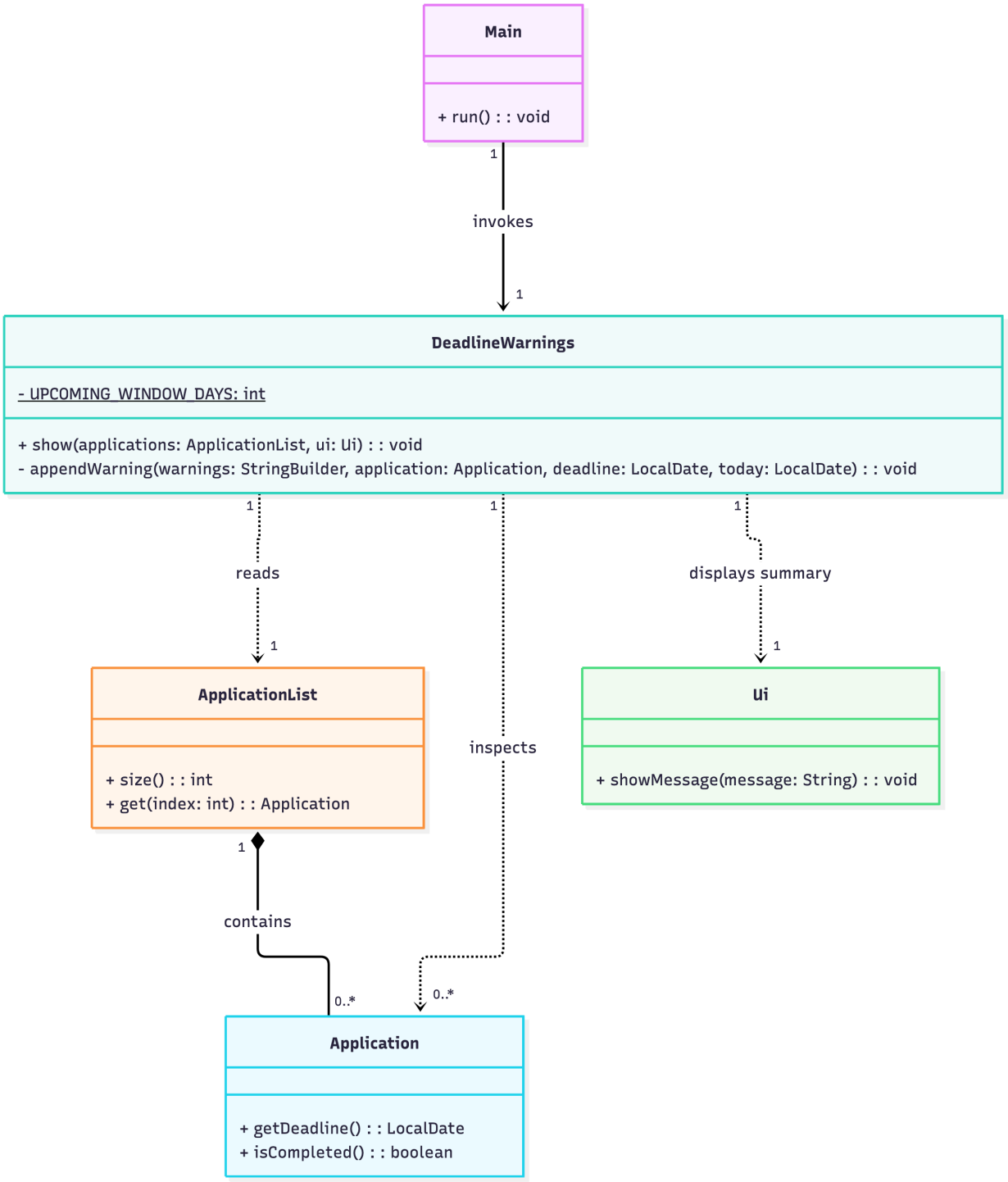
For example, if the current date is `2026-10-28`, an incomplete application with a deadline of `2026-10-27` is classified as `OVERDUE`, while one with a deadline of `2026-10-30` is classified as `UPCOMING`.

Design and implementation

The feature involves the following classes:

Class	Responsibility
`Main`	Coordinates startup and invokes the deadline check before displaying the menu.
`DeadlineWarnings`	Selects applications requiring attention and builds the warning summary.
`ApplicationList`	Provides access to the stored application records.
`Application`	Stores the optional deadline and completion flag for each application.
`Ui`	Displays the warning summary to the user.

The following class diagram shows the relationships relevant to deadline checking. Unrelated fields and methods are omitted.



Each application's deadline is represented using `LocalDate`. A `null` deadline indicates that no deadline has been recorded. `LocalDate` supports comparisons between calendar dates without requiring a time of day.

During normal startup, `Main` loads saved applications through `Storage` before `run()` begins. Within `run()`, it displays the welcome message, calls `DeadlineWarnings.show(applications, ui)`, and then displays the menu.

`DeadlineWarnings.show()` obtains the current date once using `LocalDate.now()`. It then iterates through the applications and excludes any record that:

- Has been marked as completed.
- Has no recorded deadline.

- Has a deadline later than seven days after the current date.

The remaining applications are classified using the following rules:

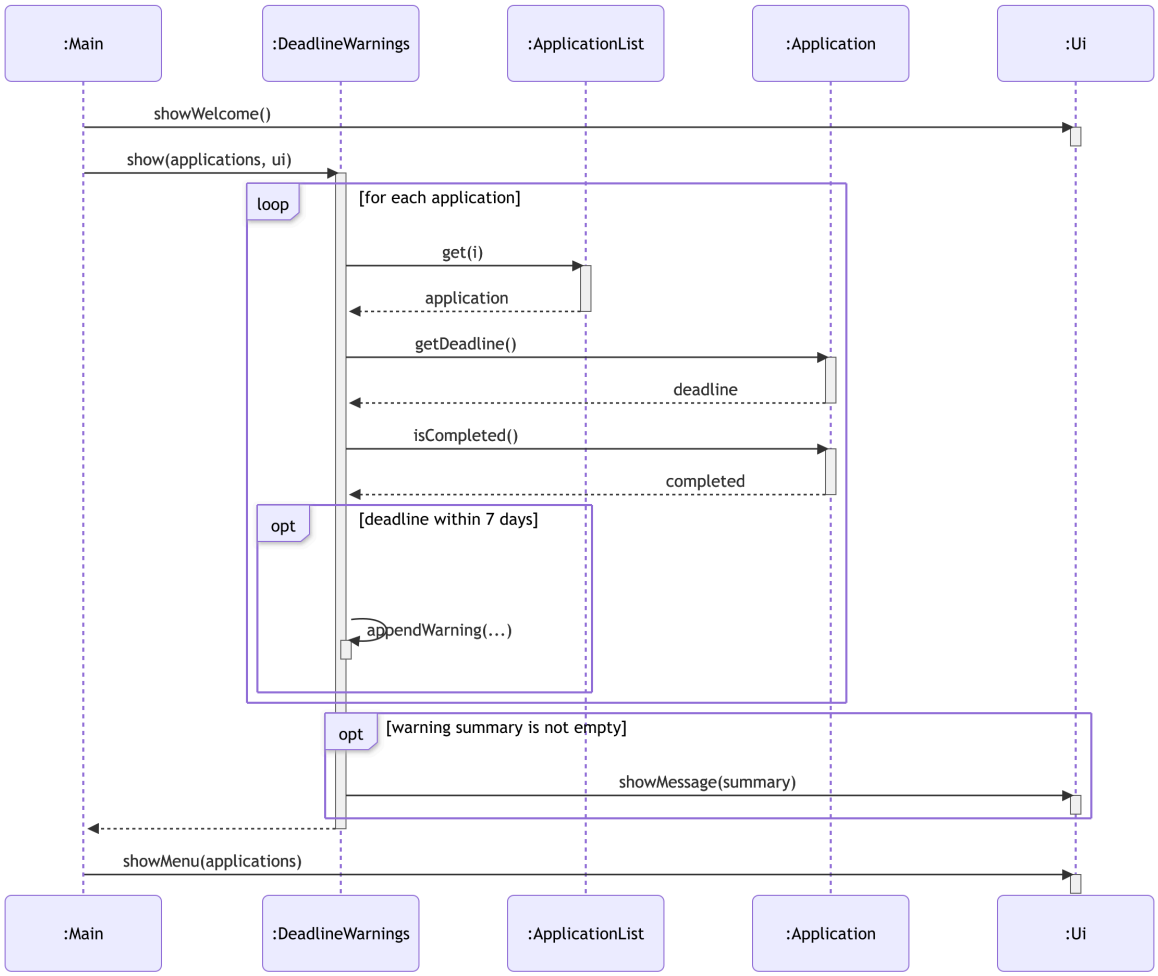
Deadline relative to today	Classification
Earlier than today	`OVERDUE`
Equal to today	`DUE TODAY`
After today, up to and including seven days away	`UPCOMING`

The seven-day boundary is inclusive. For example, when today is `2026-10-28`, a deadline of `2026-11-04` is included, while `2026-11-05` is excluded. There is no lower date limit for overdue applications.

The private `appendWarning()` method adds each warning to a `StringBuilder`. Each entry contains its classification, company, role and deadline. After checking all applications, `show()` displays the accumulated summary through a single call to `Ui.showMessage()`. If no applications qualify, it displays no warning message.

Sequence of interactions

The following sequence diagram illustrates the deadline check after saved applications have been loaded. Warning classification and formatting are summarised in a note to keep the diagram focused on interactions between objects.



Design considerations

Separate warning logic from startup coordination.
`Main` controls when the check occurs, while `DeadlineWarnings` handles selection and classification. This keeps deadline rules in one class and allows them to be changed without expanding the startup workflow. An alternative would be to place the entire check inside `Main.run()`, but that would give `Main` additional responsibility for deadline processing.

Derive warnings from application data.
Warning classifications are calculated from the deadline and current date whenever the application starts. They are not stored separately. This avoids persisting a classification such as `UPCOMING` that could become outdated as time passes.

Display one consolidated summary.
Accumulating warnings before displaying them groups all relevant deadlines together. It also keeps the output operation separate from the iteration over application records.

Current limitations and possible extensions

The check runs once at startup. Warnings do not automatically refresh after an application is added or when the date changes while JobSeek remains open. The `today` command (see Today dashboard) provides an on-demand view with the same rules, but the startup warnings themselves are still produced only once.

The upcoming window is currently fixed at seven days through `UPCOMING_WINDOW_DAYS`. It could later become configurable.

The implementation uses the system's current date directly. A future improvement could inject a `Clock` to make date-boundary tests independent of the day on which they run.

Local storage

Implemented by: Chua Jinghao, Siew Hean

Overview

The local storage feature preserves application records between JobSeek sessions. Applications are saved to `data/applications.txt` and loaded when the program starts.

The `Storage` class handles file access and conversion between application objects and their text representation. This keeps file-handling details separate from command execution and user interaction.

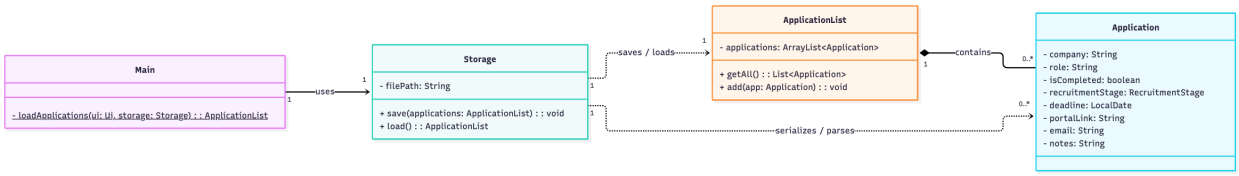
Design and implementation

The feature involves the following classes:

Class	Responsibility
`Storage`	Reads and writes application records in the local data file.
`ApplicationList`	Holds the application records in memory.
`Application`	Represents an individual application and its stored fields.
`Main`	Loads saved records during startup and reports loading errors.

`Storage` uses `data/applications.txt` by default. Its constructor also accepts a custom file path, allowing tests to use temporary files without modifying the user's saved data.

The class relationships within the storage subsystem are illustrated below:



The class diagrams show the core relationships. Save-failure protection additionally uses `Storage.ensureWritable()`, `UndoableCommand.saveOrRollback()`, `CommandHistory.peek()` and the temporary `ApplicationListSnapshot` described above. Earlier activity and sequence illustrations omit these failure branches; the workflows in the text define save-before-confirmation behavior and rollback.

File format

The file uses UTF-8 encoding and contains two header lines followed by application records. Each record contains eight tab-separated fields:

Field	Representation
Company	Escaped text
Role	Escaped text
Completion flag	`true` or `false`
Deadline	ISO date such as `2026-11-03`, or an empty field
Recruitment stage	Escaped text such as `Applied`; an empty field is read as `Applied`
Company portal link	Escaped URL text, or an empty field
Contact email	Escaped email address, or an empty field
Notes	Escaped freeform text, or an empty field

An empty deadline, portal link, email, or notes field represents an application where that optional attribute has not been recorded.

Backslashes, tabs, newlines and carriage returns within text fields are escaped (`\\`, `\t`, `\n`, `\r`) before saving. During loading, `splitFields()` decodes these escape sequences. This prevents characters inside company, role, or notes from being mistaken for field or record separators.

A line beginning with `#` is treated as a comment only if it contains no tab separators. A tab-separated record beginning with `#Acme` is therefore parsed normally, including in legacy four-field and five-field files.

Backward compatibility: Files written by earlier versions have four fields (legacy, without stage) or five fields (prior stage format). `Storage.load()` accepts four-field, five-field, and modern eight-field records. Four-field records receive the default `Applied` stage, and existing `Pending` values automatically migrate to `Applied`. Missing portal link, email, and notes default to empty strings. It also accepts older column headers such as `status`. The next save rewrites the file in the full eight-field format with `stage` as the fifth column.

Saving applications

Commands that modify application data call `Storage.save(applications)` to persist the updated list.

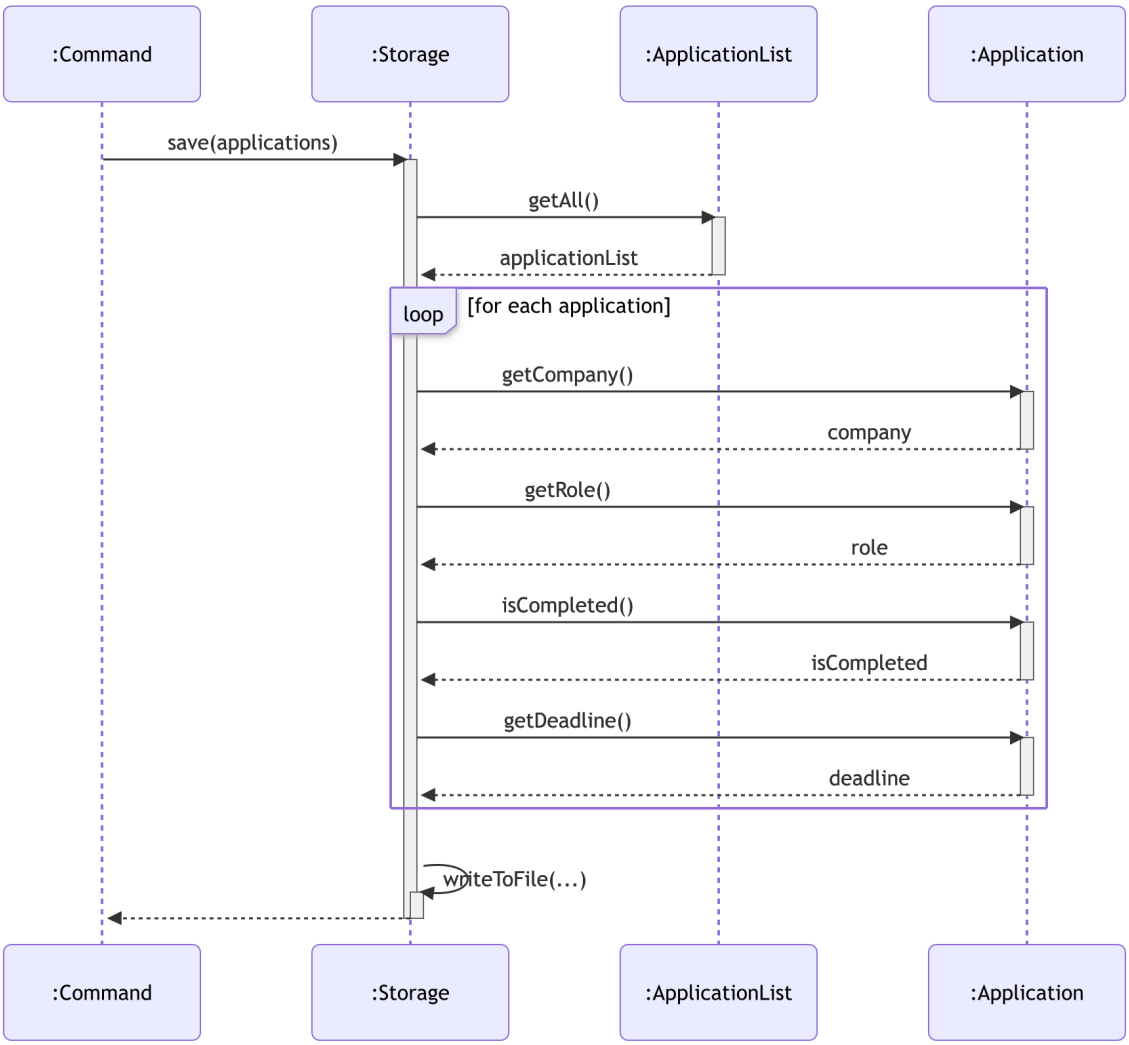
The save operation performs the following steps:

1. Converts the configured file path into a `Path`.
2. Creates missing parent directories, if required.
3. Adds the title and column headers.
4. Converts each application into a tab-separated record.
5. Writes the complete contents using UTF-8 encoding to a temporary file in the same directory, then moves it over the data file (an atomic move where the file system supports it).

Writing straight over the data file would truncate it before any new data is written. Writing to a temporary file first leaves the original untouched if that write fails. Once the temporary file is complete, storage requests an atomic move: where supported, the replacement occurs as one operation, without exposing a partly replaced file.

If the file system reports `AtomicMoveNotSupportedException`, storage falls back to a regular move with replacement for compatibility. This fallback is **not atomic**: if the move fails or the process crashes during replacement, the original file is not guaranteed to remain intact. Atomic replacement also does not, by itself, guarantee that saved data survives a power failure. If writing or moving reports an error, storage attempts to delete the temporary file and passes the `IOException` to the caller, whose in-memory change is rolled back. That rollback does not repair a data file damaged during the non-atomic fallback.

The sequence of interactions during a save operation is shown below:



The operation rewrites the file using the current list. Consequently, applications removed from the list are also removed from the saved data.

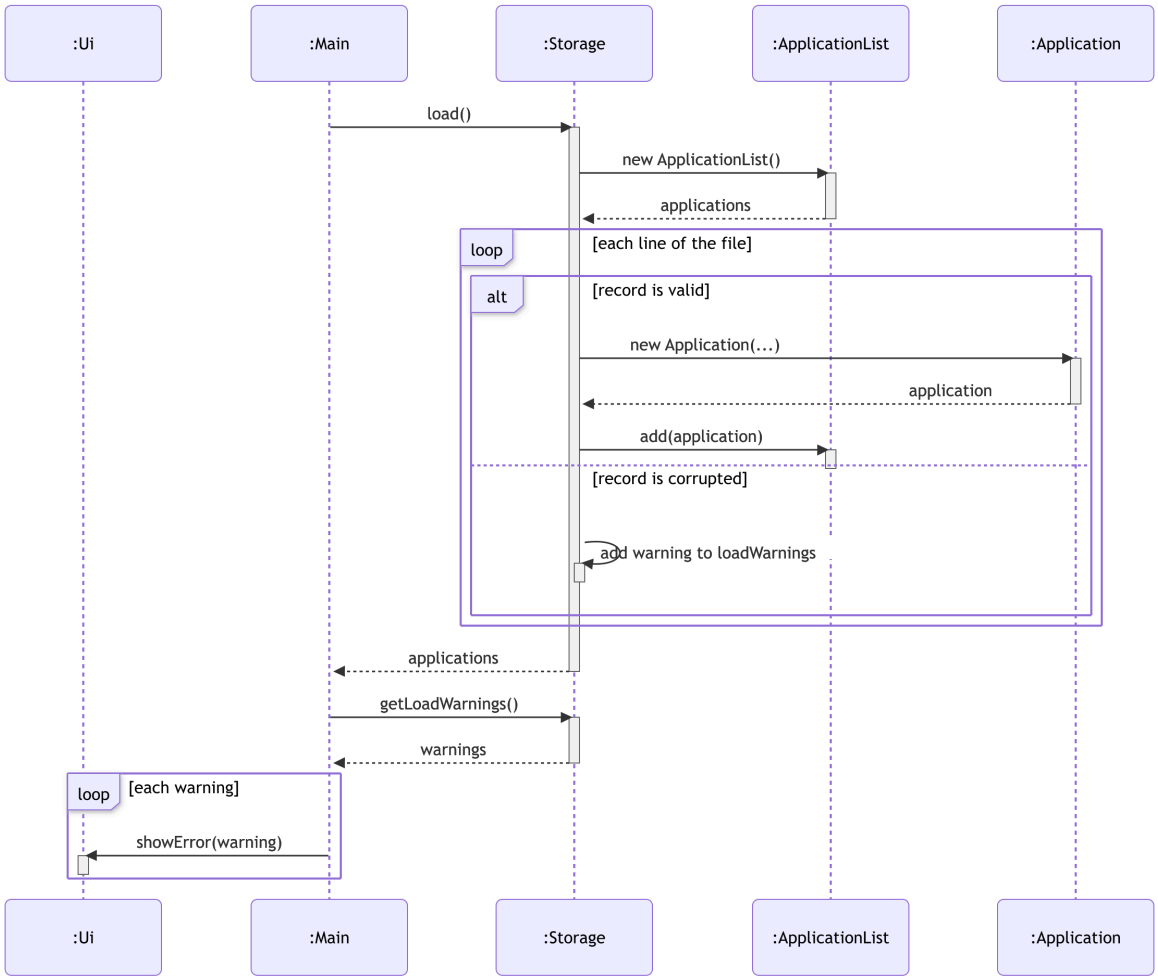
Loading applications

During normal startup, `Main` calls `Storage.load()` before entering the main interaction loop.

If the file does not exist, `load()` logs an informational event via `java.util.logging.Logger` and returns an empty `ApplicationList`. Otherwise, it reads the file and processes each line. Blank lines, comment lines beginning with `#`, and recognised header lines are filtered out.

In adherence to the Single Level of Abstraction Principle (SLAP), `Storage.load()` delegates the parsing and validation of individual lines to a private helper, `parseRecord(int lineIndex, String line)`. For each record, `Storage` verifies that there are four fields (legacy format), five fields (legacy status format), or eight fields (modern format with optional portal link, email, notes). It ensures the completion flag is boolean (`true` or `false`), company and role are non-empty, optional deadlines conform to `LocalDate` format, and recruitment stage maps to a valid `RecruitmentStage`. Valid records are instantiated as `Application` objects and added to the list. Any line that fails validation is skipped, and an informative warning naming the line number is recorded in `Storage.getLoadWarnings()`.

The following sequence diagram shows successful loading of an existing file. File-reading details are summarised in a note:



Error handling

JobSeek recovers gracefully from corrupted lines instead of discarding the entire file. If a line has an invalid escape sequence, the wrong number of fields, an invalid completion flag, an empty company or role, an invalid deadline or an unknown stage, `parseRecord()` skips that line, logs a warning, and records a warning message. All other valid lines in the file are loaded without interruption.

After loading, `Main.loadApplications()` displays all recorded warnings via `Ui.showError()`. An unrecoverable I/O failure (such as file access permission denied or invalid UTF-8) causes `load()` to throw an `IOException`, which `Main` catches to inform the user and initialize with an empty list. Storage keeps saving blocked until a subsequent load succeeds. Before executing an undoable command or undo, `Main` calls `Storage.ensureWritable()` so the empty fallback list cannot be modified. `Storage.save()` repeats this check to protect direct callers. A confirmed missing file remains a normal, writable first launch; recoverable line-level warnings do not activate the block.

Skipped lines are not rewritten until the next save, and the next save writes only the records that loaded. To avoid losing them, `load()` sets a `backupPending` flag whenever it skipped a line, and `save()` first copies the original file to `<path>.bak` (once, replacing any older backup) before writing. `Main.loadApplications()` tells the user about this after showing the warnings. A leading byte-order mark is stripped from the first line before header detection.

Saving errors (`IOException`) propagate up to `Main.run()`, which catches them and presents a clear error message via `Ui.showError()`. Modifying commands call `UndoableCommand.saveOrRollback()` before displaying success. If saving fails, the command reverses its in-memory change and clears its changed

flag, leaving prior history intact. `UndoCommand` instead peeks at the latest history entry and captures an `ApplicationListSnapshot` before undoing it. If saving fails, the snapshot restores fields, ordering and original object identities; the history entry is removed only after a successful save. This preserves references held by earlier commands and allows failed undo operations to be retried. This rollback protects in-memory state; file replacement has the file-system-dependent guarantees described in Saving applications.

Design considerations

Human-readable text format.
The tab-separated format is simple to inspect and uses Java's standard file APIs. An alternative would be JSON, which could represent additional fields more explicitly but would require a different serialisation approach.

Save the complete list.
Rewriting the complete list keeps persistence straightforward because additions, updates and deletions use the same save operation. The trade-off is that each save processes every application.

Separate persistence from application logic.
Commands request a save without implementing file encoding or parsing themselves. Changes to the storage format can therefore be concentrated in `Storage`.

Possible improvements

Recruitment stages are persisted in the final column. The loader also accepts legacy files whose column header is `status`. A crash at exactly the wrong moment can still leave an unused `applications-*.tmp` file in the `data` folder; it is harmless and can be deleted.

The backup is overwritten the next time a load skips lines. A future version could keep several timestamped backups.

Error handling and logging architecture

JobSeek employs a multi-tiered error handling and defensive programming strategy across user input, domain operations, file persistence, and internal invariants:

1. Domain vs. Operational Exception Separation

- **`JobSeekException`**: A custom checked domain exception (package `jobseek.exception`) representing anticipated business and user-facing validation errors (e.g., malformed typed commands, invalid filter criteria, out-of-bounds application indices in direct commands). These are caught by callers or `Main.run()` and rendered cleanly via `Ui.showError()` without exposing technical stack traces or crashing the program.
- **`IOException`**: Narrowed from generic `Exception` to specific file system failures. Handles operational storage operations (such as missing read/write permissions or disk errors). During `Storage.load()`, recoverable line-level corruption is caught locally, logging a diagnostic warning and recording a load warning, allowing remaining intact records to load successfully. Only fatal I/O failures bubble up to `Main.run()`.

2. Defensive Programming and Assertions

JobSeek utilizes Java `assert` statements throughout constructors and internal command execution to enforce preconditions, postconditions, and internal class invariants:

- **Preconditions**: Verifying non-null arguments and valid index states (`assert targetIndex >= 0`, `assert list != null`, `assert history != null`).
- **Postconditions & State Invariants**: Verifying application fields are never blank or null after updates, and list indices remain consistent. Assertions are enabled during automated testing (`./gradlew test`) and development to trap logic regressions immediately.

3. Diagnostic Logging Architecture

JobSeek utilizes Java's standard `java.util.logging.Logger` across critical architectural layers to provide transparent operational observability without polluting user-facing terminal output:

- **`Main` (`INFO`)**: Logs high-level application lifecycle events, such as startup, storage initialization, fallback to menu dispatch, and graceful shutdown.
- **`Storage` (`INFO` & `WARNING`)**: Logs persistence file creation and successful save operations at `INFO`. Logs corrupted or malformed lines that fail loading validation at `WARNING` with the specific line number.
- **`CommandParser` (`FINE`)**: Traces parsed typed command tokens and input dispatches for development-level diagnosis.

Product scope

Target user profile

University students who apply for several internships and graduate roles at the same time, and who:

- are comfortable typing and prefer a keyboard to a mouse;
- manage their applications alone on their own computer;
- are comfortable running a Java program from a terminal.

Value proposition

Application information is usually spread across company portals, emails, notes and memory. JobSeek keeps one personal, human-editable record of every application's company, role, recruitment stage, deadline and completion state. Students can capture an application in one line, see what needs attention today, find applications quickly when the list grows, and undo mistakes.

User Stories

Version	As a ...	I want to ...	So that I can ...
v1.0	student beginning to track applications	add a new job application	keep an opportunity in one central record
v1.0	new user	see usage instructions	refer to them when I forget how to use the application
v1.0	job seeker	view detailed information of a specific application	inspect all fields including recruitment stage and deadlines in a structured card
v1.0	job seeker	mark an application as completed	track which applications have concluded their recruitment cycle
v1.0	job seeker	unmark a completed application	reopen an application if the recruitment process resumes or if marked by mistake
v2.0	job seeker	find applications by company or role	locate an application without scanning the full list
v2.0	job seeker	view applications alphabetically	compare applications by company and role more easily
v2.0	job seeker	filter applications by active or completed status	focus on ongoing applications without distraction from completed ones
v2.0	student	edit the recorded details of an application	keep information accurate when details change or I make a mistake
v2.0	student		